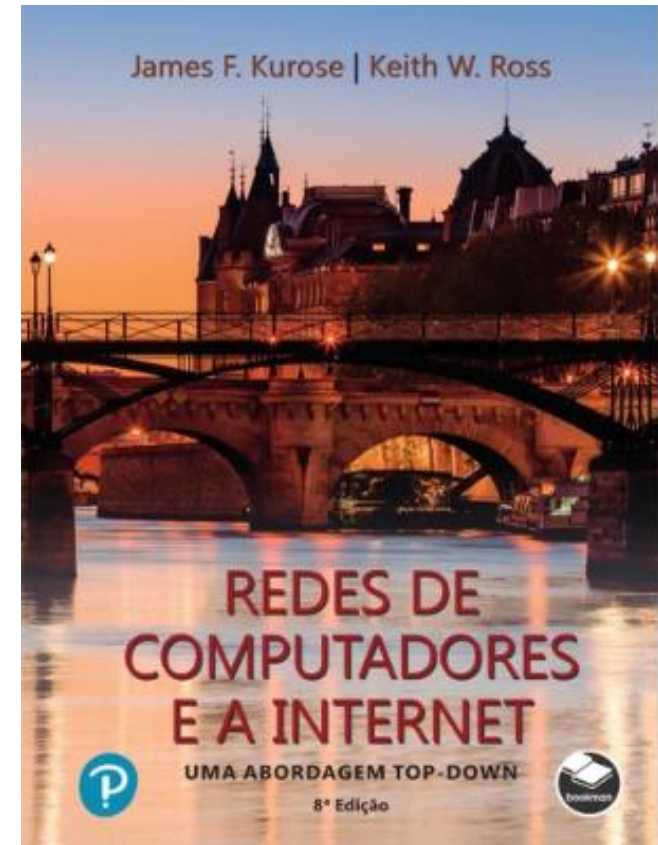


Capítulo 3

Camada de Transporte

All material copyright 1996-2023
J.F Kurose and K.W. Ross, All Rights Reserved



*Redes de Computadores e
a Internet: Uma
abordagem Top-Down*

8ª edição

Jim Kurose, Keith Ross

Pearson & Bookman, 2021

Capítulo 3: Camada de Transporte

Nosso objetivo:

- entender os princípios atrás dos serviços da camada de transporte:
 - multiplexação/demultiplexação
 - transferência confiável de dados
 - controle de fluxo
 - controle de congestionamento
- aprender sobre os protocolos da camada de transporte da Internet:
 - UDP: transporte não orientado a conexões
 - TCP: transporte confiável orientado a conexões
 - Controle de congestionamento do TCP

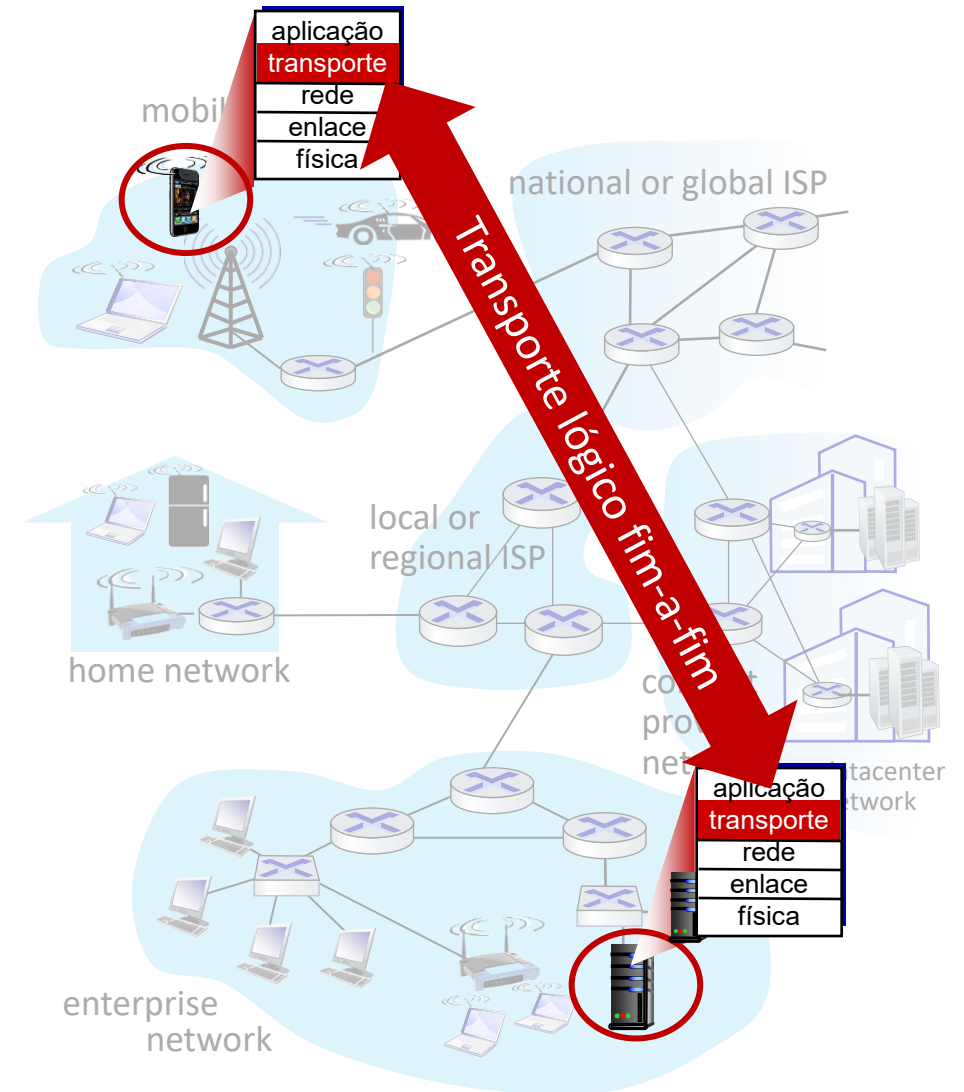
Capítulo 3: roteiro

- Serviços da camada de transporte
- Multiplexação e demultiplexação
- Transporte sem conexões: UDP
- Princípios da transferência confiável
- Transporte orientado a conexões: TCP
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte



Serviços e protocolos de transporte

- fornecem *comunicação lógica* entre processos de aplicação executando em diferentes hospedeiros
- os protocolos de transporte são executados nos sistemas finais:
 - transmissor: quebra as mensagens da aplicação em **segmentos**, repassa-os para a camada de rede
 - receptor: remonta as mensagens a partir dos segmentos, repassa-as para a camada de aplicação
- dois protocolos de transporte disponíveis para as aplicações Internet:
 - TCP e UDP



Serviços e protocolos das camadas de Transporte x rede



Analogia doméstica:

12 crianças na casa de Ana enviando cartas para 12 crianças na casa de Bill

- hospedeiros = casas
- processos = crianças
- mensagens da apl. = cartas nos envelopes

Serviços e protocolos das camadas de Transporte x rede

- *camada de transporte:* comunicação lógica entre os **processos**
 - depende de e estende serviços da camada de rede
- *camada de rede:* comunicação lógica entre **hospedeiros**

Analogia doméstica:

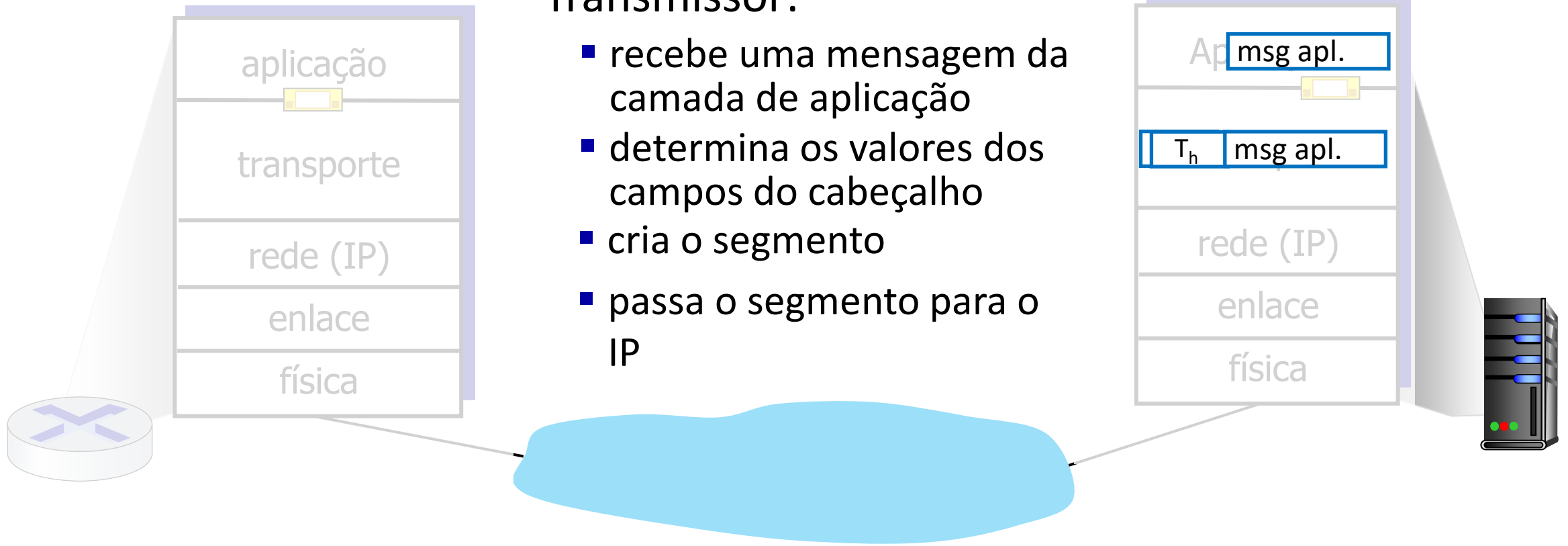
12 crianças na casa de Ana enviando cartas para 12 crianças na casa de Bill

- hospedeiros = casas
- processos = crianças
- mensagens da apl. = cartas nos envelopes

Ações da Camada de Transporte

Transmissor:

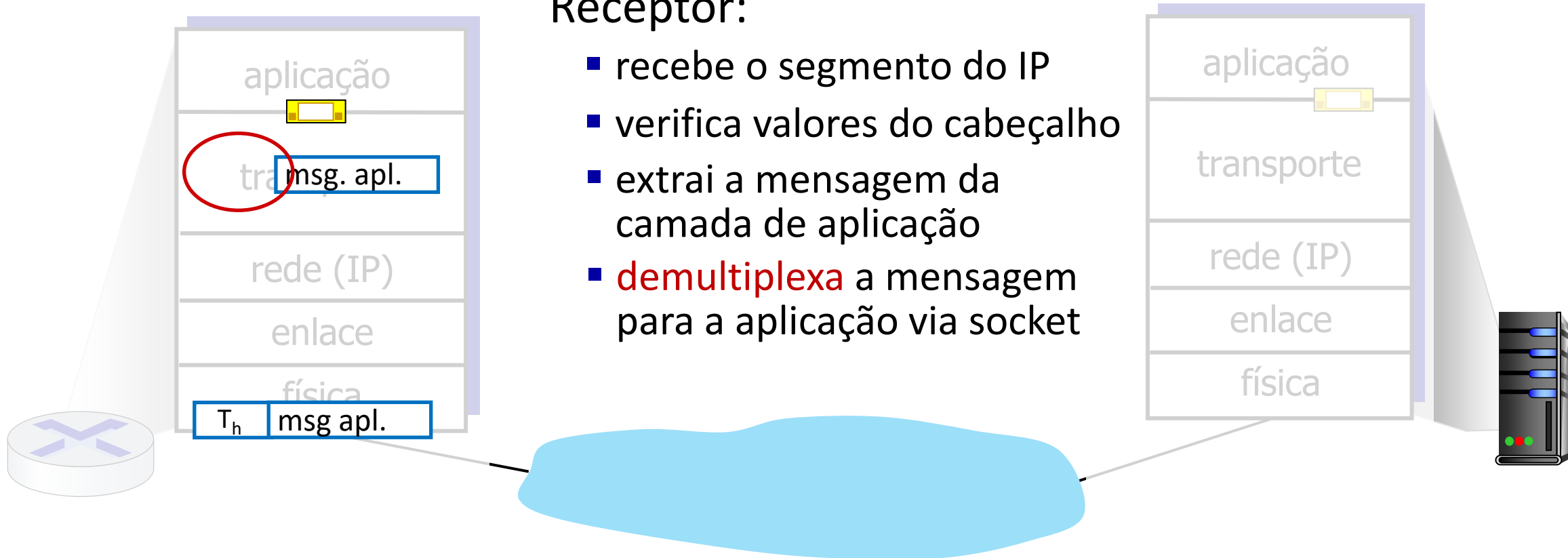
- recebe uma mensagem da camada de aplicação
- determina os valores dos campos do cabeçalho
- cria o segmento
- passa o segmento para o IP



Ações da Camada de Transporte

Receptor:

- recebe o segmento do IP
- verifica valores do cabeçalho
- extrai a mensagem da camada de aplicação
- **demultiplexa** a mensagem para a aplicação via socket



Protocolos da camada de transporte Internet

■ **TCP:** *Transmission Control Protocol*

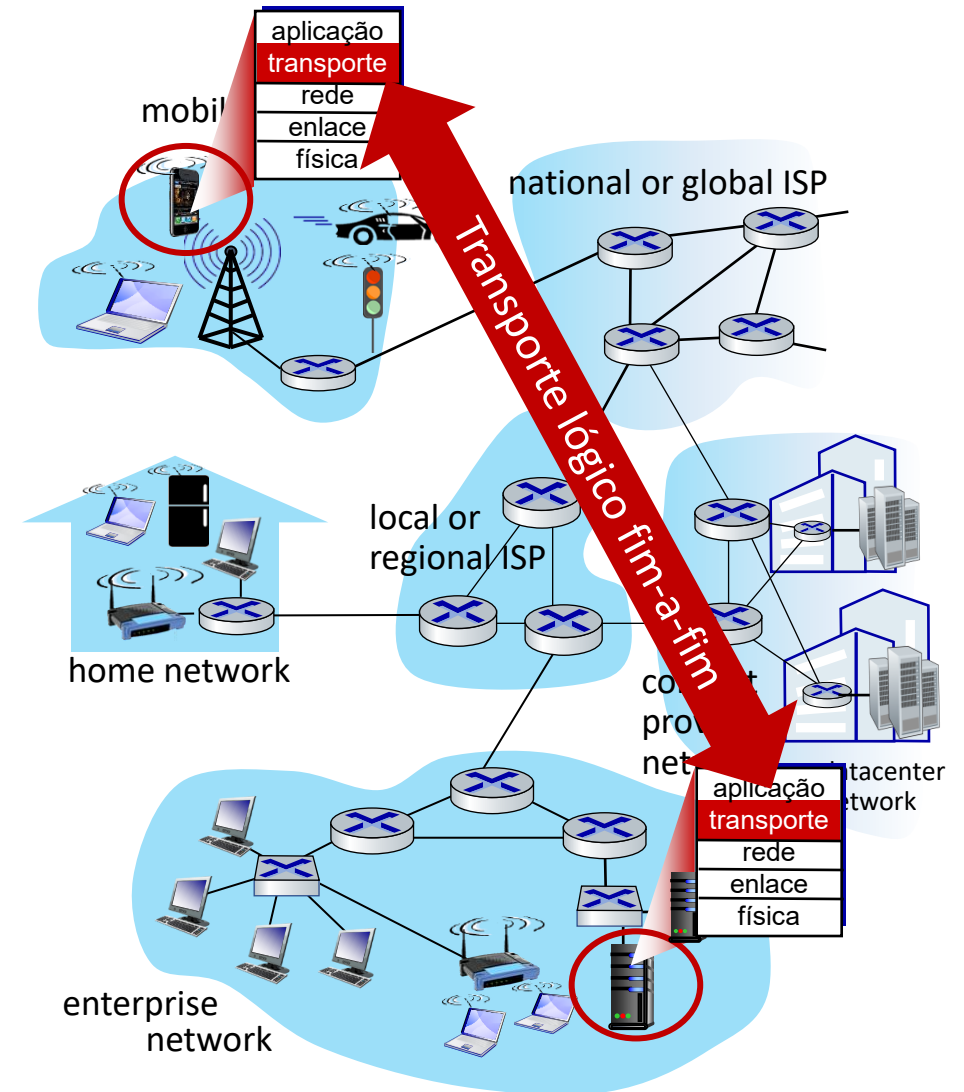
- entrega confiável, ordenada
- controle de congestionamento
- controle de fluxo
- estabelecimento de conexão (“*setup*”)

■ **UDP:** *User Datagram Protocol*

- entrega não confiável, não ordenada: UDP
- extensão sem “gorduras” do “melhor esforço” do IP

■ serviços *não* disponíveis:

- garantias de atraso máximo
- garantias de largura de banda mínima



Capítulo 3: roteiro

- Serviços da camada de transporte
- **Multiplexação e demultiplexação**
- Transporte sem conexões: UDP
- Princípios da transferência confiável
- Transporte orientado a conexões: TCP
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte



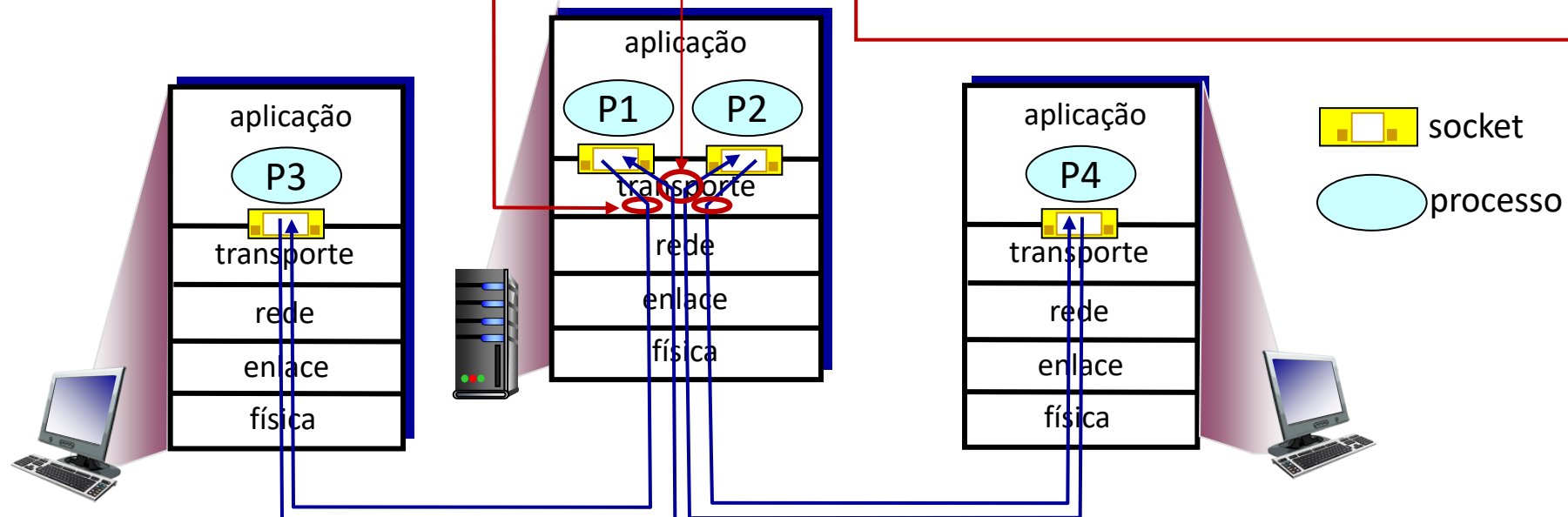
Multiplexação/demultiplexação

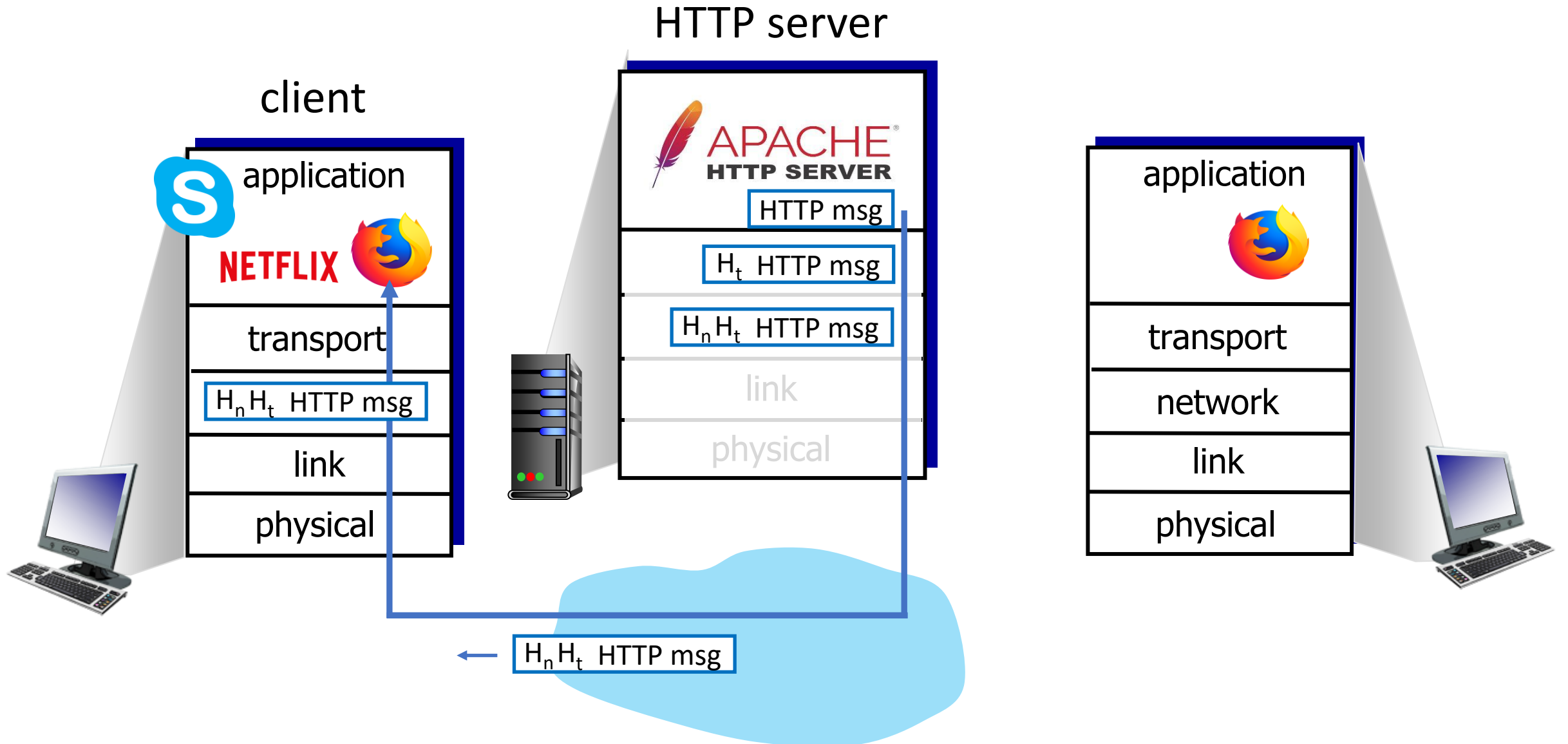
multiplexação no transmissor:

reúne dados de muitos sockets, adiciona o cabeçalho de transporte (usado posteriormente para a demultiplexação)

Demultiplexação no receptor:

Usa info do cabeçalho para entregar os segmentos recebidos aos sockets corretos

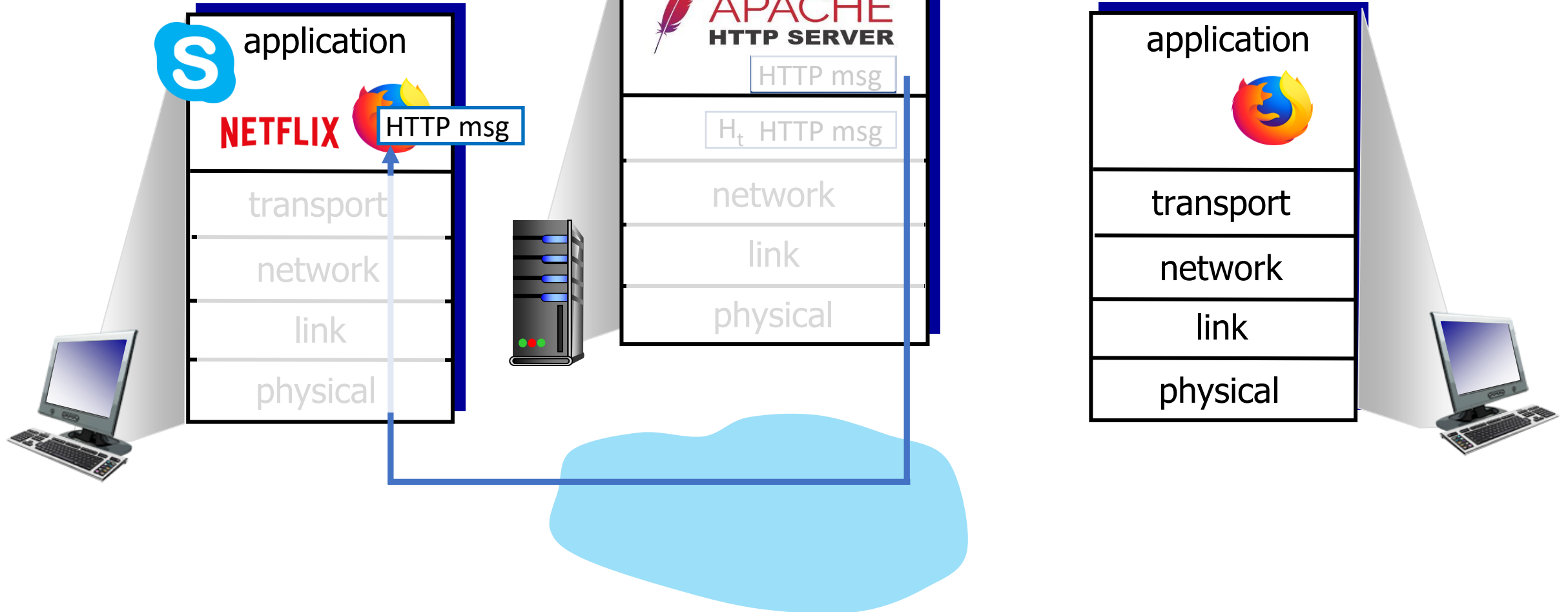


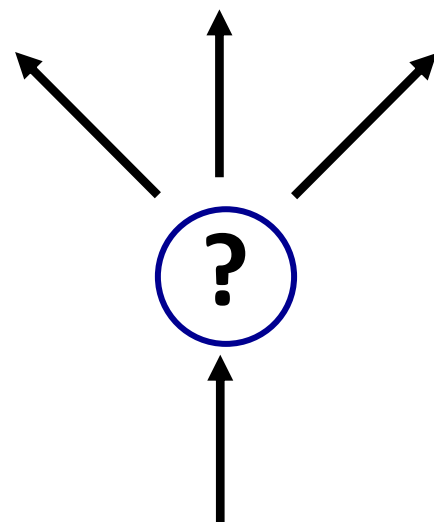




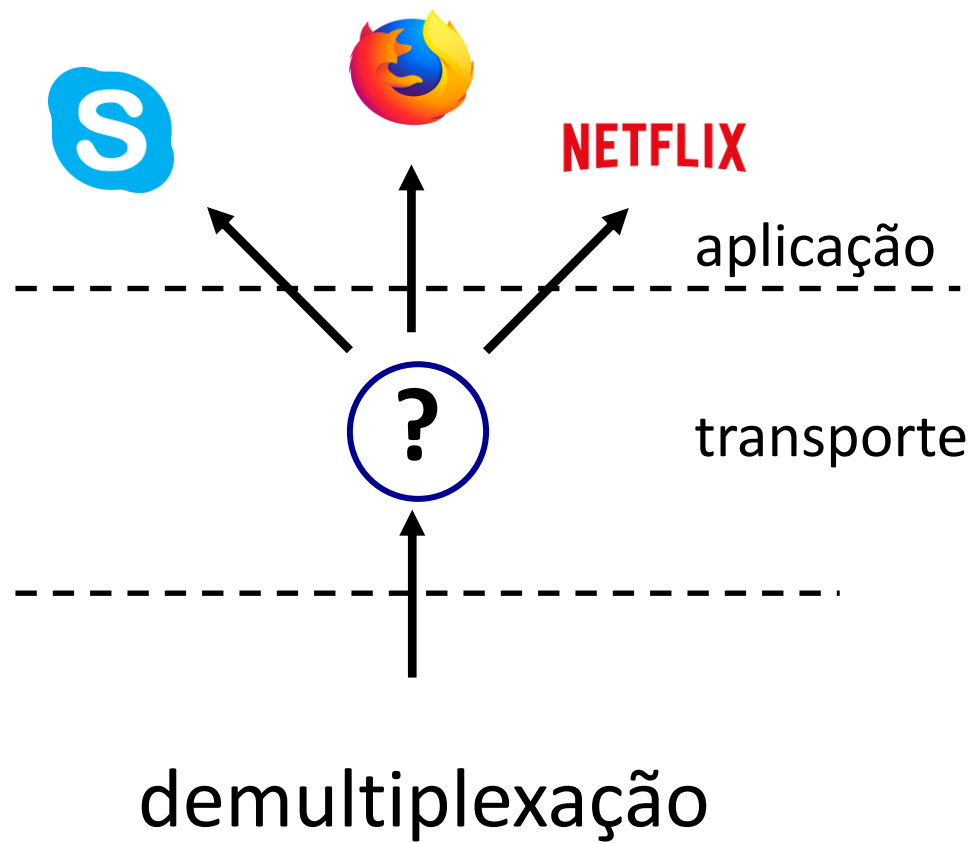
Q: como a camada de transporte sabia que deveria entregar a mensagem ao navegador Firefox e não ao processo Netflix ou ao processo Skype?

cliente





demultiplexação





Demultiplexação

AIRFRANCE 

ECONOMY 



AIRFRANCE 

SKY
PRIORITY™



TSA Pre✓



Transportation
Security
Administration

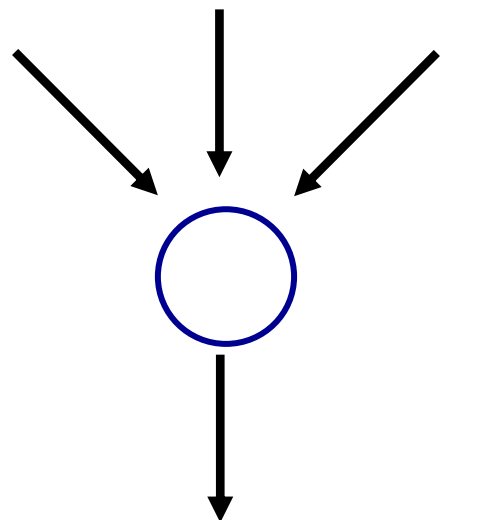
[tsa.gov](https://www.tsa.gov)

Main
Checkpoint

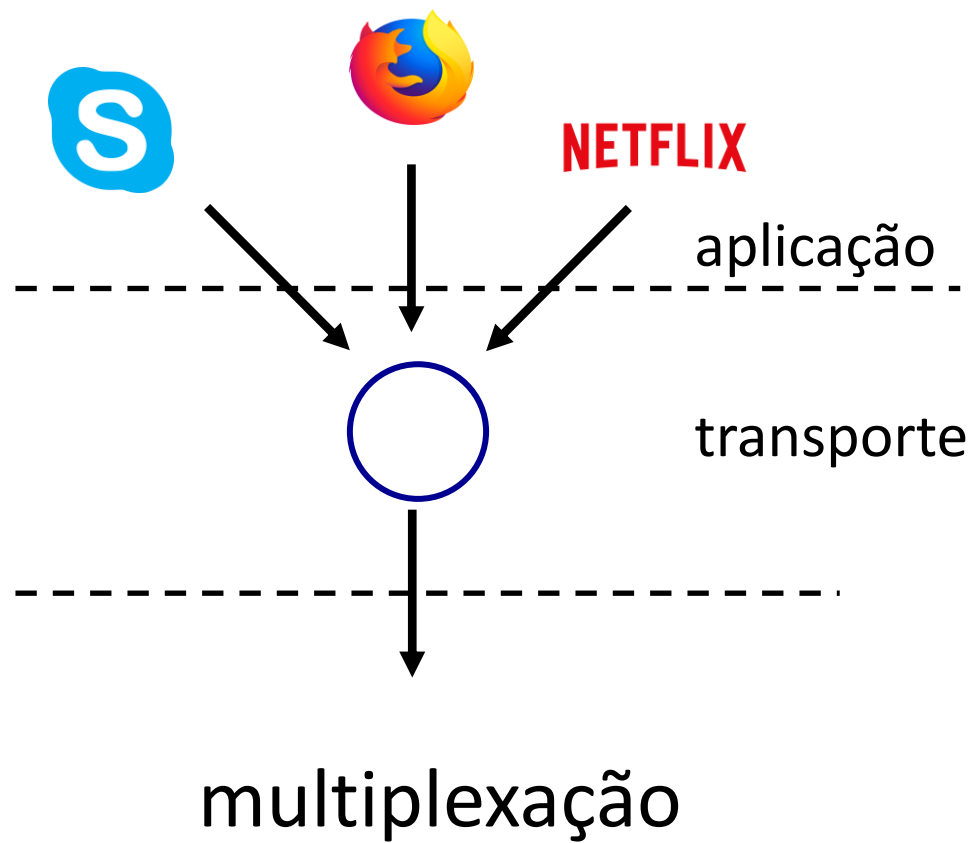


Transportation
Security
Administration

[tsa.gov](https://www.tsa.gov)



multiplexação





Multiplexação

Como funciona a demultiplexação

- computador recebe os datagramas IP
 - cada datagrama possui os endereços IP da origem e do destino
 - cada datagrama transporta um segmento da camada de transporte
 - cada segmento possui números das portas origem e destino
- O hospedeiro usa os **endereços IP** e **os números das portas** para direcionar o segmento ao socket apropriado



Demultiplexação não orientada a conexões

Lembrete:

- socket criado possui número de porta **local ao host**:

```
DatagramSocket mySocket1 = new  
DatagramSocket(12534);
```

- r ao criar um datagrama para enviar para um socket UDP, deve especificar:

- m endereço IP de destino
- m número da porta de destino

quando o hospedeiro recebe o segmento UDP:

- m verifica no. da porta de destino no segmento
- m encaminha o segmento UDP para o socket com aquele no. de porta



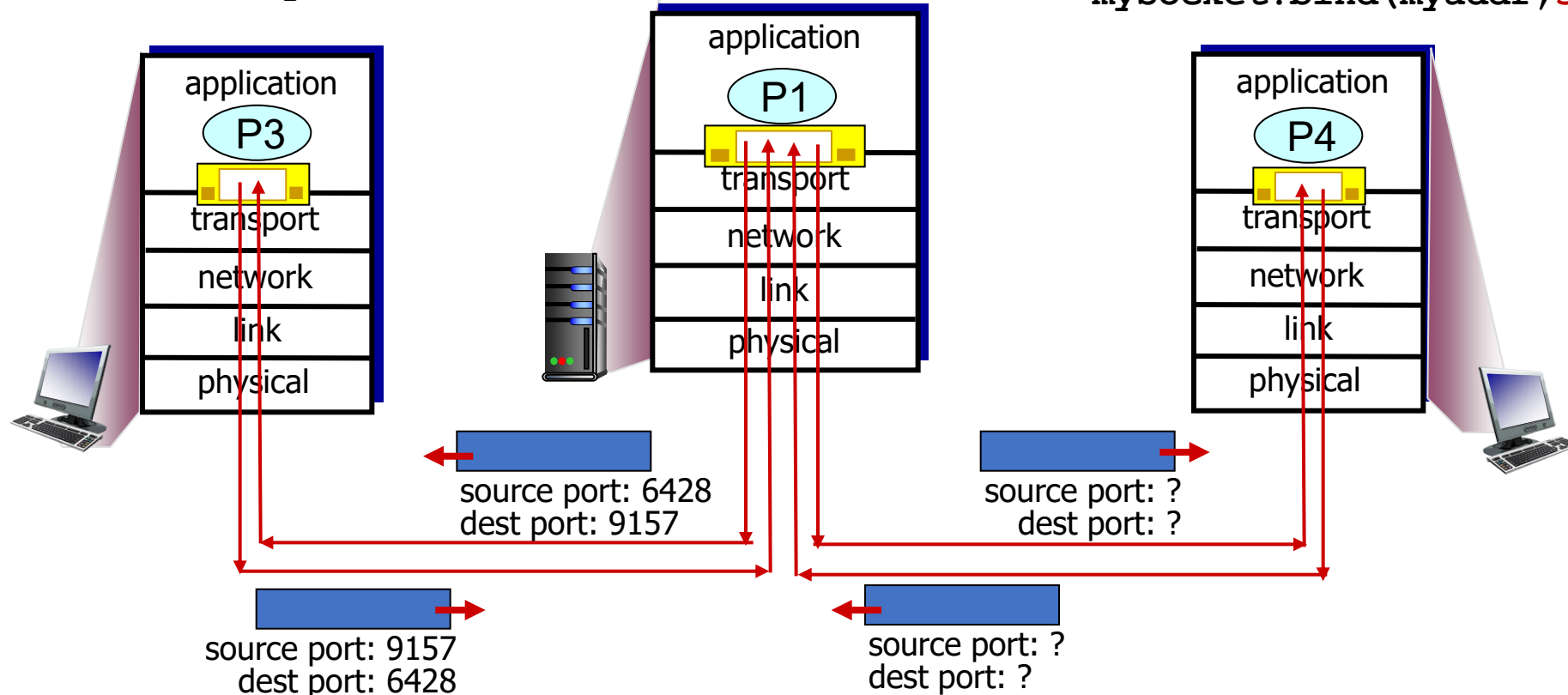
datagramas IP com **mesmo no. de porta destino**, mas diferentes endereços IP origem e/ou números de porta origem podem ser encaminhados para o **mesmo socket** no destino

Demultiplexação não orientada a conexões: exemplo

```
mySocket =  
    socket(AF_INET, SOCK_DGRAM)  
mySocket.bind(myaddr, 6428);
```

```
mySocket =  
    socket(AF_INET, SOCK_DGRAM)  
mySocket.bind(myaddr, 9157);
```

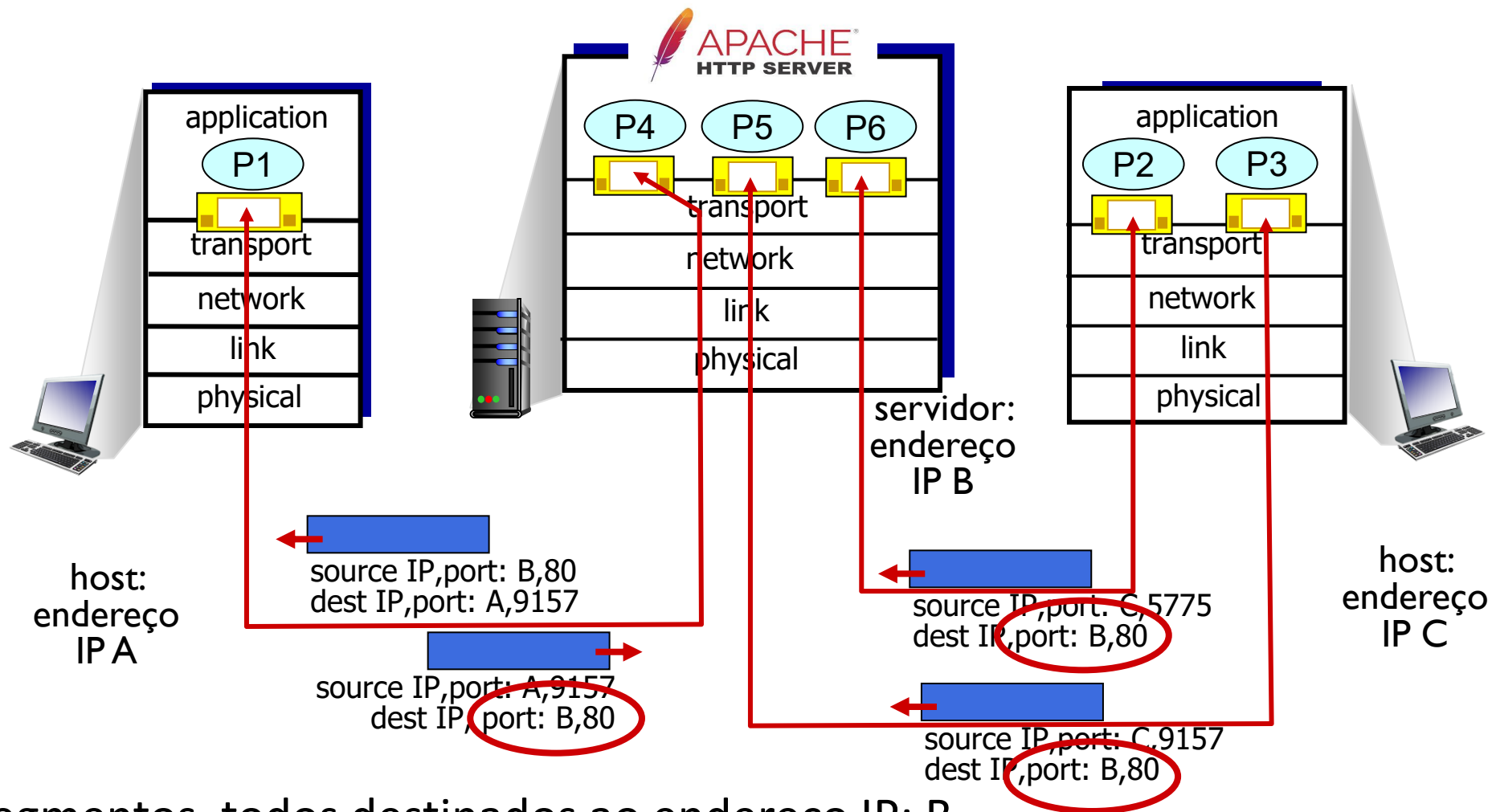
```
mySocket =  
    socket(AF_INET, SOCK_DGRAM)  
mySocket.bind(myaddr, 5775);
```



Demultiplexação Orientada a Conexões

- Socket TCP identificado pela **quádrupla**:
 - endereço IP origem
 - número da porta origem
 - endereço IP destino
 - número da porta destino
 - Servidor pode dar suporte a muitos sockets TCP simultâneos:
 - cada socket é identificado pela sua própria quádrupla
 - cada socket está associado a um cliente diferente
- r Demultiplexação: receptor usa **todos os quatro valores** para direcionar o segmento para o socket apropriado

Demultiplexação Orientada a Conexões: exemplo



Três segmentos, todos destinados ao endereço IP: B, porta dest: 80 são demultiplexados para *diferentes* sockets

Resumo

- Multiplexação, demultiplexação: baseada nos valores dos campos de cabeçalho do segmento, datagrama.
- **UDP:** demultiplexa usando (apenas) o número da porta de destino
- **TCP:** demultiplexa usando a quádrupla: endereços IP e números de porta da origem e do destino
- Multiplexação/demultiplexação ocorre em *todas* as camadas

Capítulo 3: roteiro

- Serviços da camada de transporte
- Multiplexação e demultiplexação
- **Transporte sem conexões: UDP**
- Princípios da transferência confiável
- Transporte orientado a conexões: TCP
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte



UDP: *User Datagram Protocol*

- Protocolo de transporte da Internet mínimo, “sem gorduras”,
- Serviço “melhor esforço”, segmentos UDP podem ser:
 - perdidos
 - entregues à aplicação fora de ordem
- *sem conexão*:
 - não há saudação inicial entre o remetente e o receptor UDP
 - tratamento independente para cada segmento UDP

— Por quê existe o UDP? —

- r elimina estabelecimento de conexão (que adiciona atraso RTT)
- r simples: não mantém “estado” da conexão nem no remetente, nem no receptor
- r cabeçalho de segmento reduzido
- r Não há controle de congestionamento:
 - m UDP pode transmitir tão rápido quanto desejado (e possível)
 - m funciona mesmo diante de congestionamento

UDP: *User Datagram Protocol*

- Uso do UDP:
 - aplicações de fluxos (*streaming*) multimídia (tolerante a perdas, sensível à taxa de transmissão)
 - DNS
 - SNMP
 - HTTP/3
- caso seja necessária transferência confiável sobre o UDP (ex., HTTP/3):
 - adiciona confiabilidade necessária na camada de aplicação
 - adiciona controle de congestionamento na camada de aplicação

UDP: *User Datagram Protocol* [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel

ISI

28 August 1980

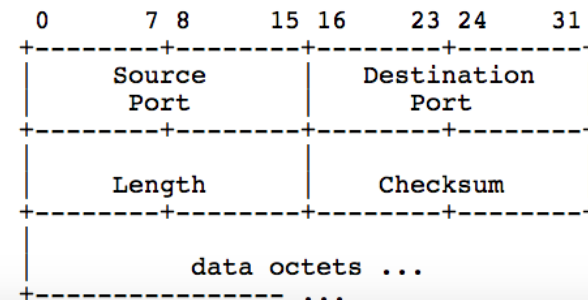
User Datagram Protocol

Introduction

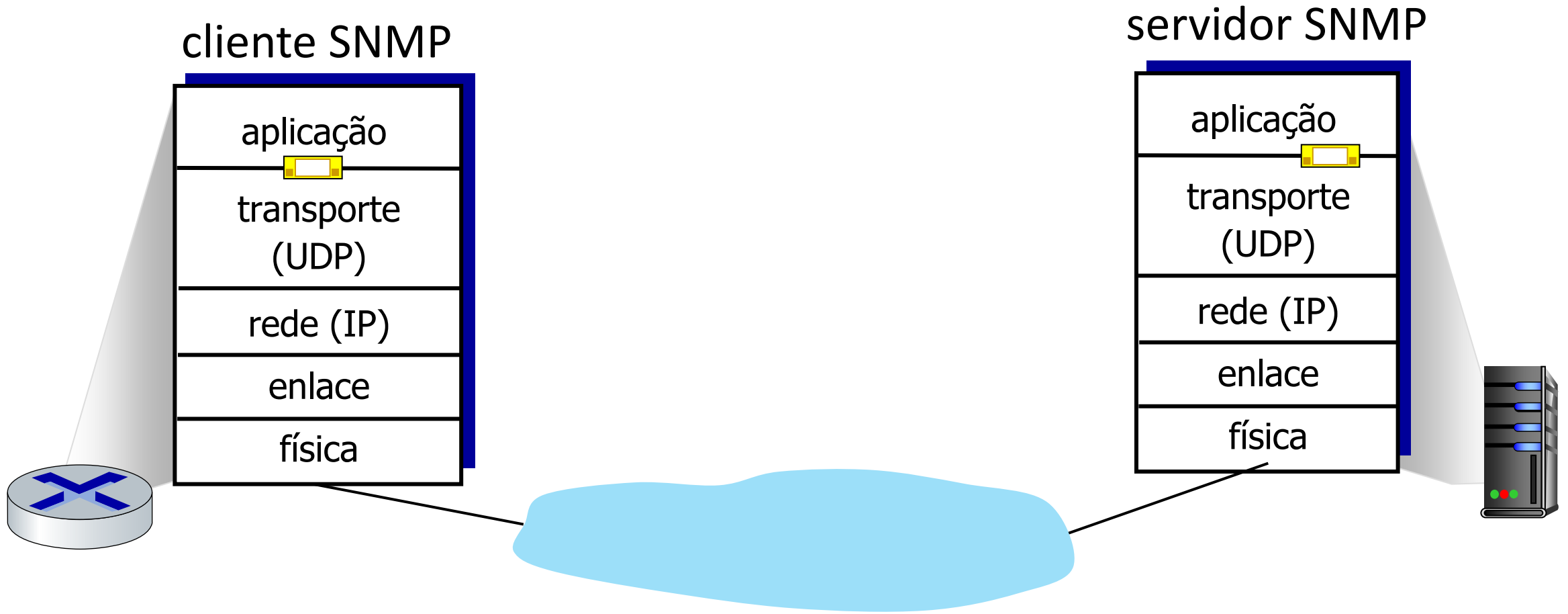
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

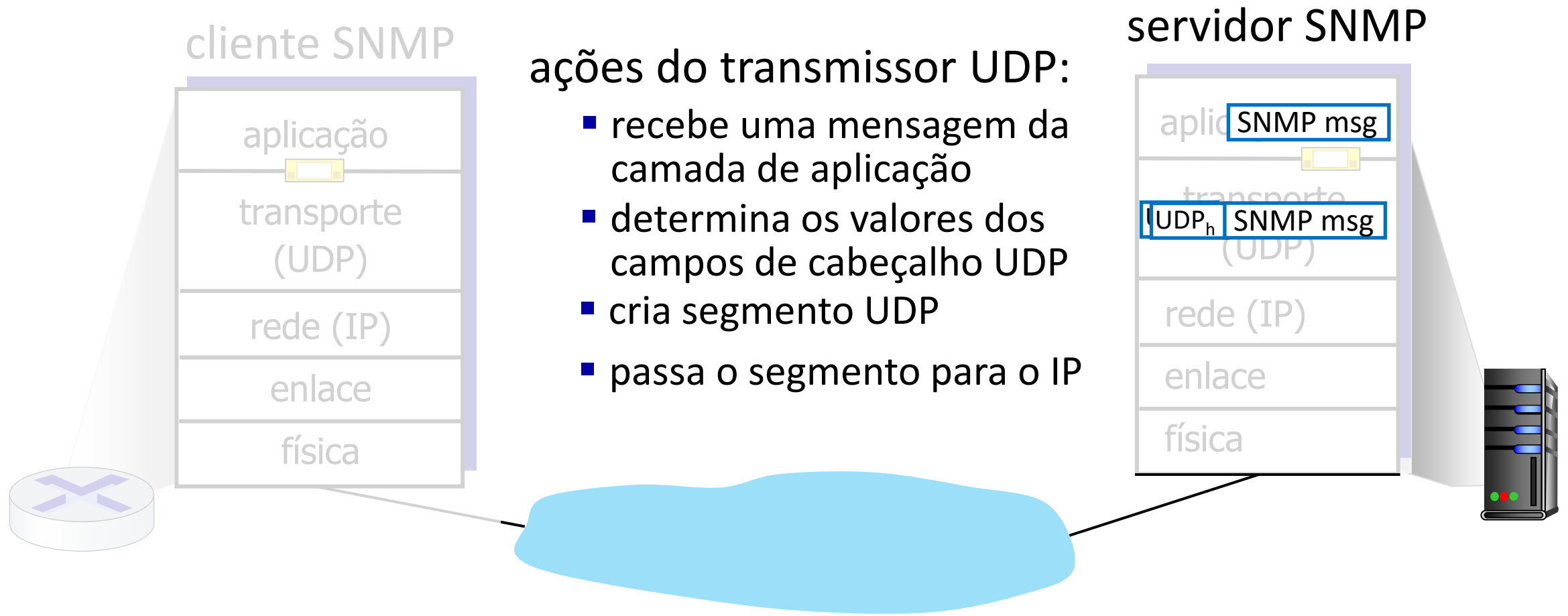
Format



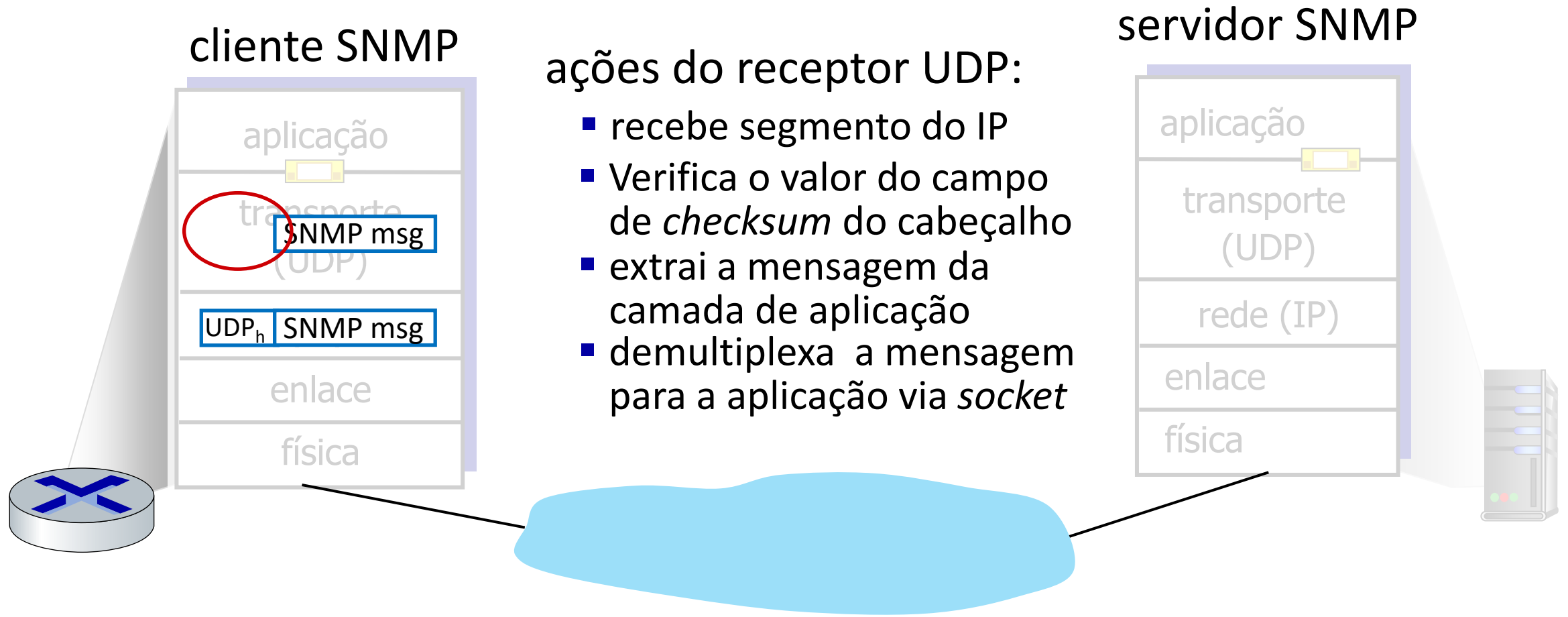
UDP: Ações da Camada de Transporte



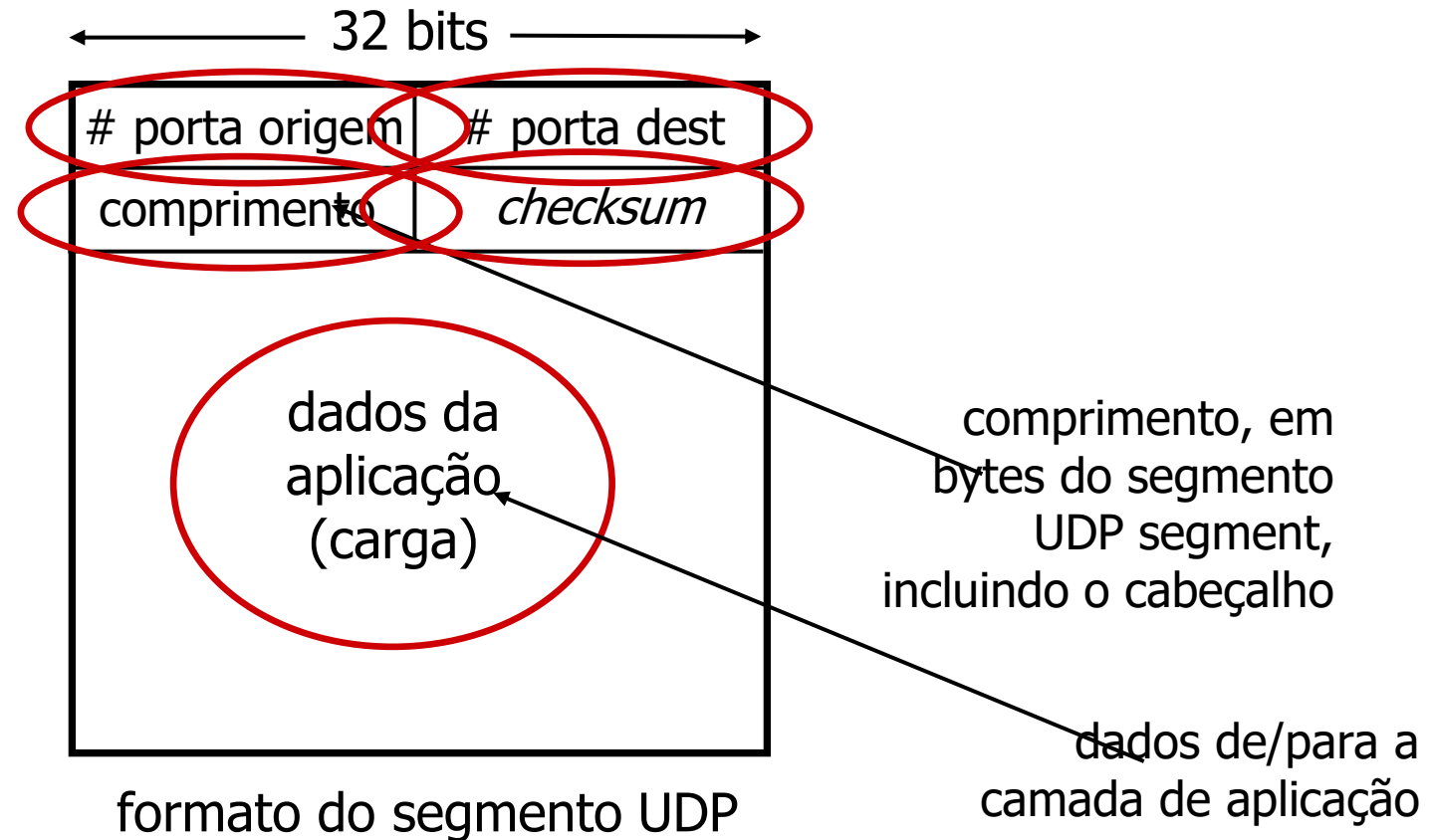
UDP: Ações da Camada de Transporte



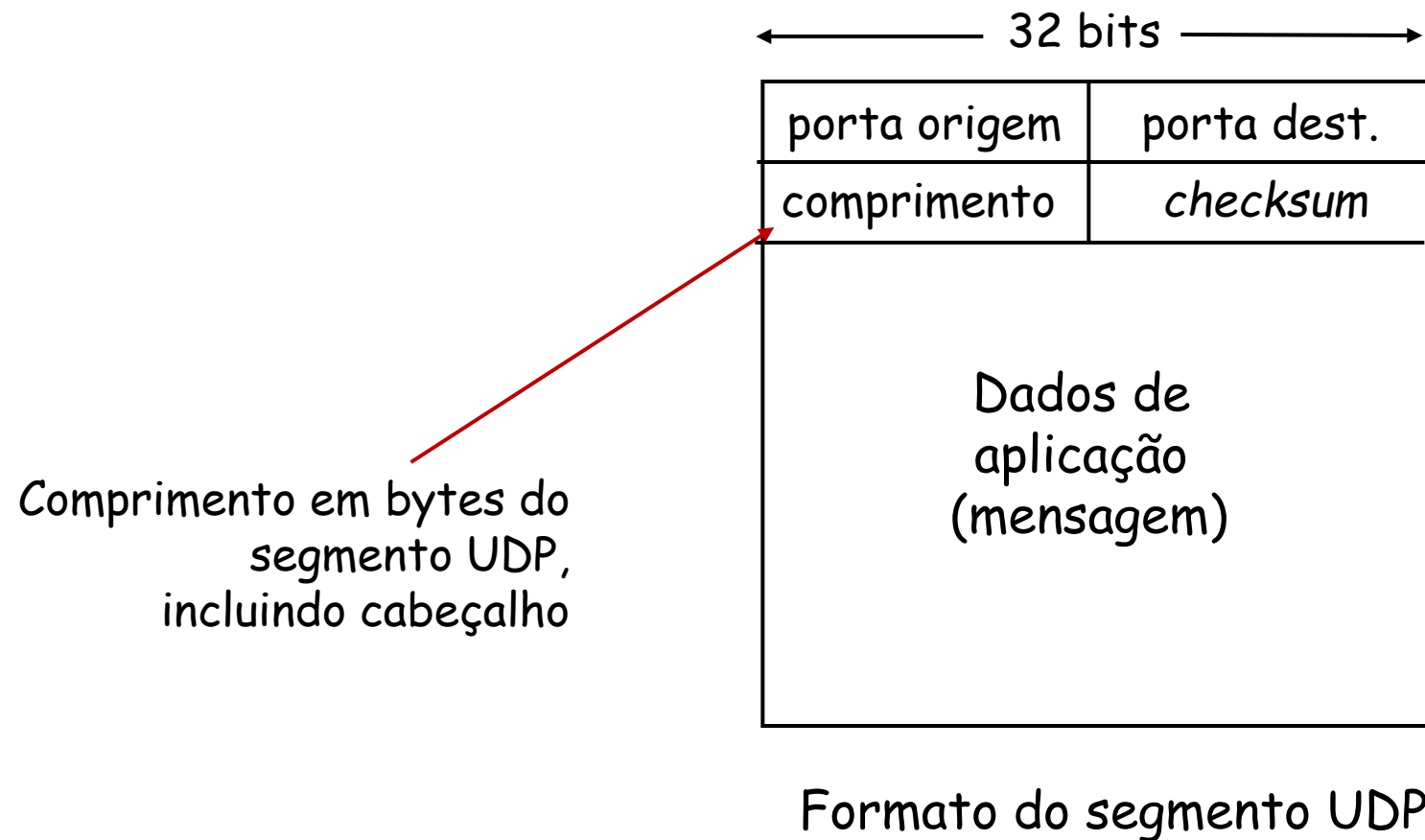
UDP: Ações da Camada de Transporte



UDP: Cabeçalho do segmento

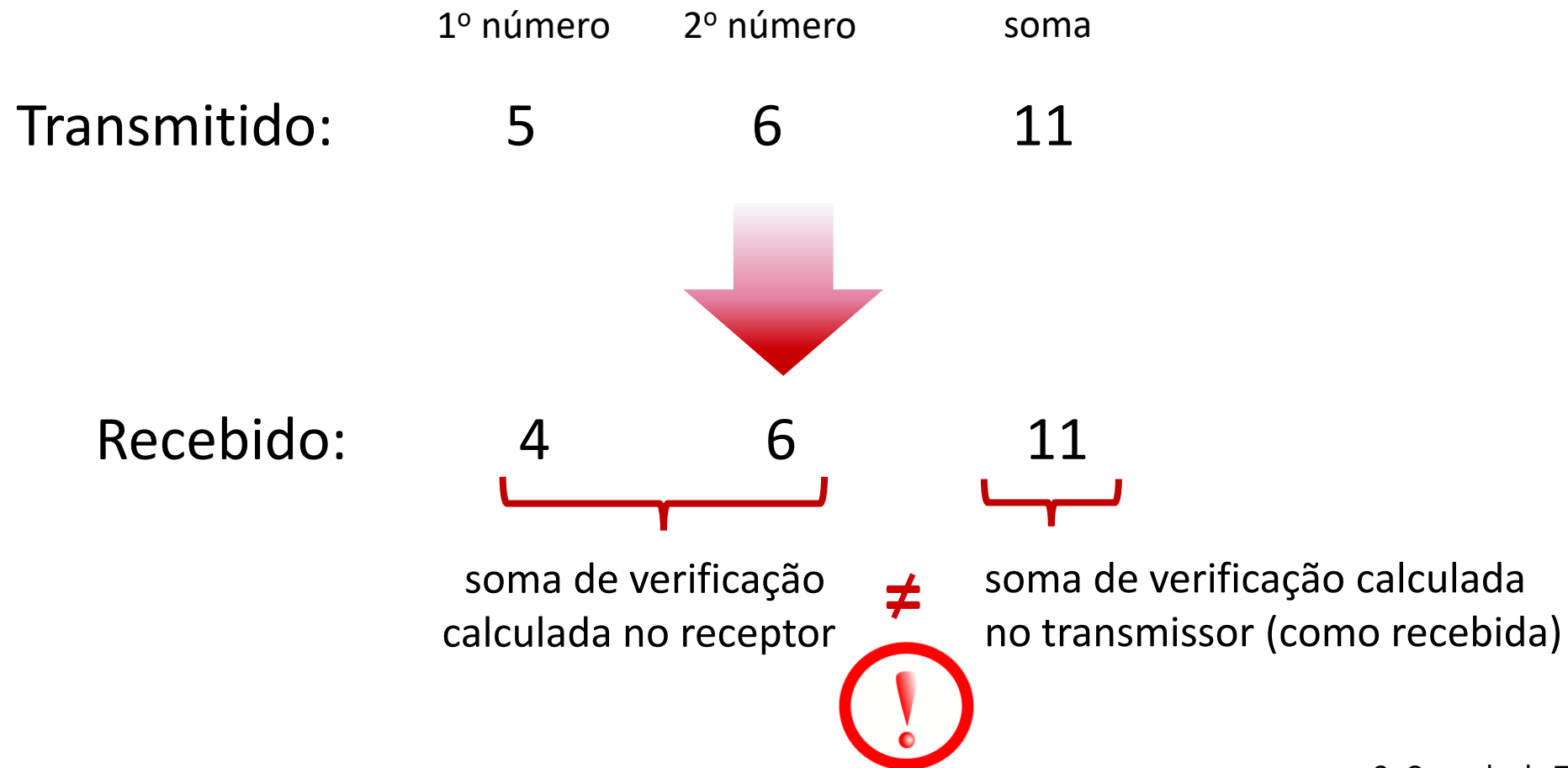


UDP: Cabeçalho do segmento



Soma de Verificação (*checksum*) UDP

Objetivo: detectar “erros” (ex.: bits trocados) no segmento transmitido



Soma de Verificação (*checksum*) UDP

Objetivo: detectar “erros” (ex.: bits trocados) no segmento transmitido

transmissor:

- trata conteúdo do segmento UDP (incluindo campos do cabeçalho UDP e endereços IP) como sequência de inteiros de 16-bits
- ***checksum***: soma (adição usando complemento de 1) do conteúdo do segmento
- transmissor coloca o complemento do *valor da soma* no campo *checksum* do UDP

receptor:

- calcula *checksum* do segmento recebido
- verifica se o *checksum* calculado bate com o valor recebido:
 - NÃO - erro detectado
 - SIM - nenhum erro detectado. *Mas ainda pode ter erros? Veja depois*

UDP: Resumo

- protocolo “sem gorduras”:
 - segmentos podem ser perdidos, entregues fora de ordem
 - serviço melhor esforço: “envia e torce que tudo dê certo”
- UDP tem pontos positivos:
 - não necessita de estabelecimento de conexão (*setup/handshaking*)
 - não incorrendo em um RTT adicional
 - pode funcionar quando o serviço de rede estiver comprometido
 - ajuda com a confiabilidade (*checksum*)
- podem ser adicionadas funcionalidades acima do UDP na camada de aplicação (ex.: HTTP/3)

Capítulo 3: roteiro

- Serviços da camada de transporte
- Multiplexação e demultiplexação
- Transporte sem conexões: UDP
- **Princípios da transferência confiável**
- Transporte orientado a conexões: TCP
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte



Princípios de Transferência confiável de dados

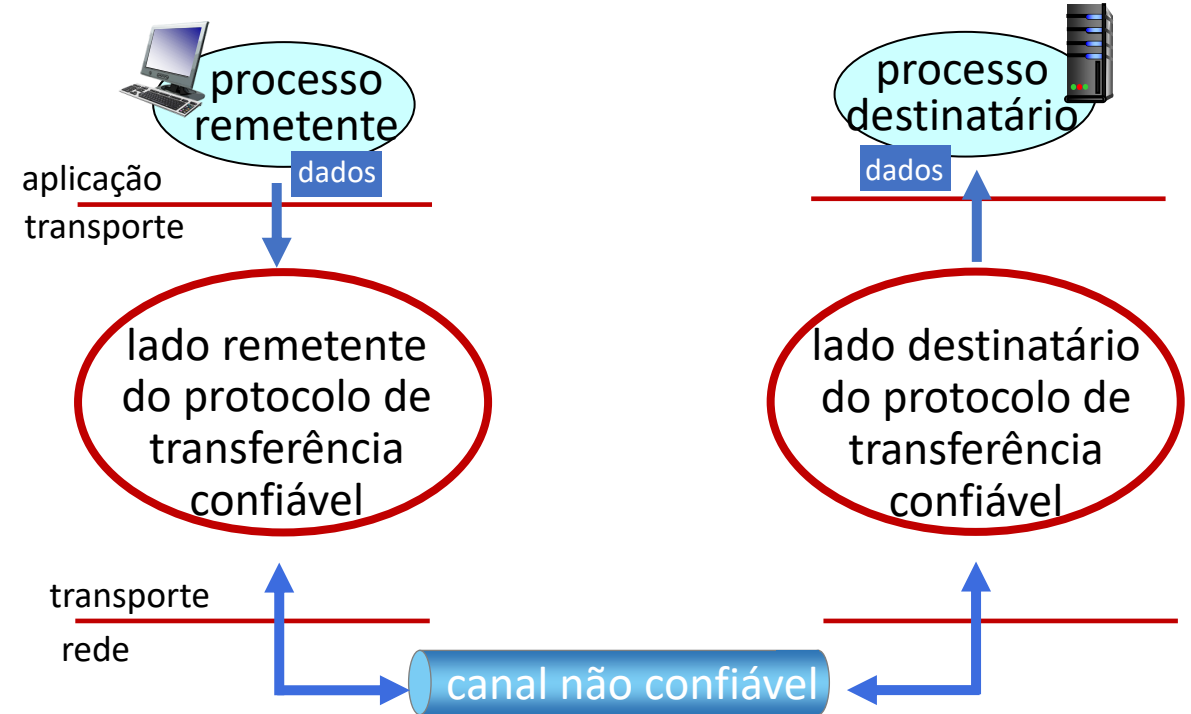
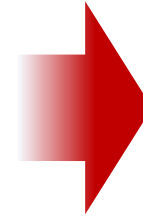


abstração do serviço confiável

Princípios de Transferência confiável de dados



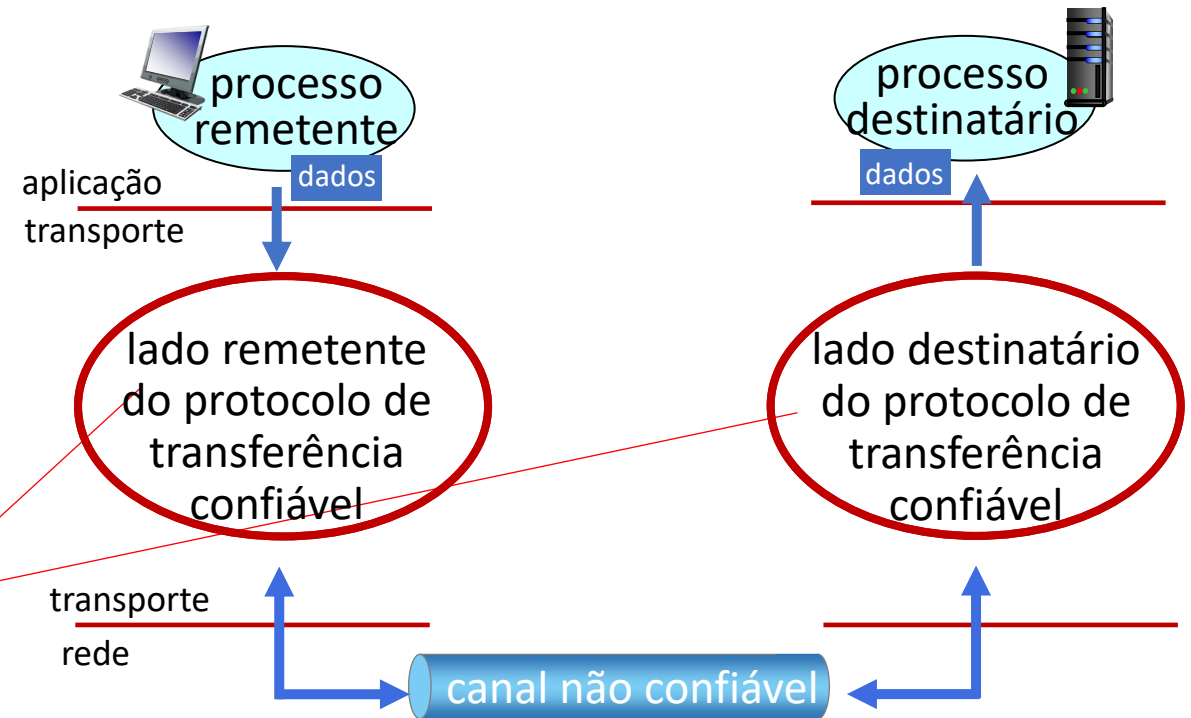
abstração do serviço confiável



implementação do serviço confiável

Princípios de Transferência confiável de dados

Complexidade do protocolo de transferência confiável de dados dependerá (fortemente) das características do canal não confiável (perde, corrompe, reordena os dados?)

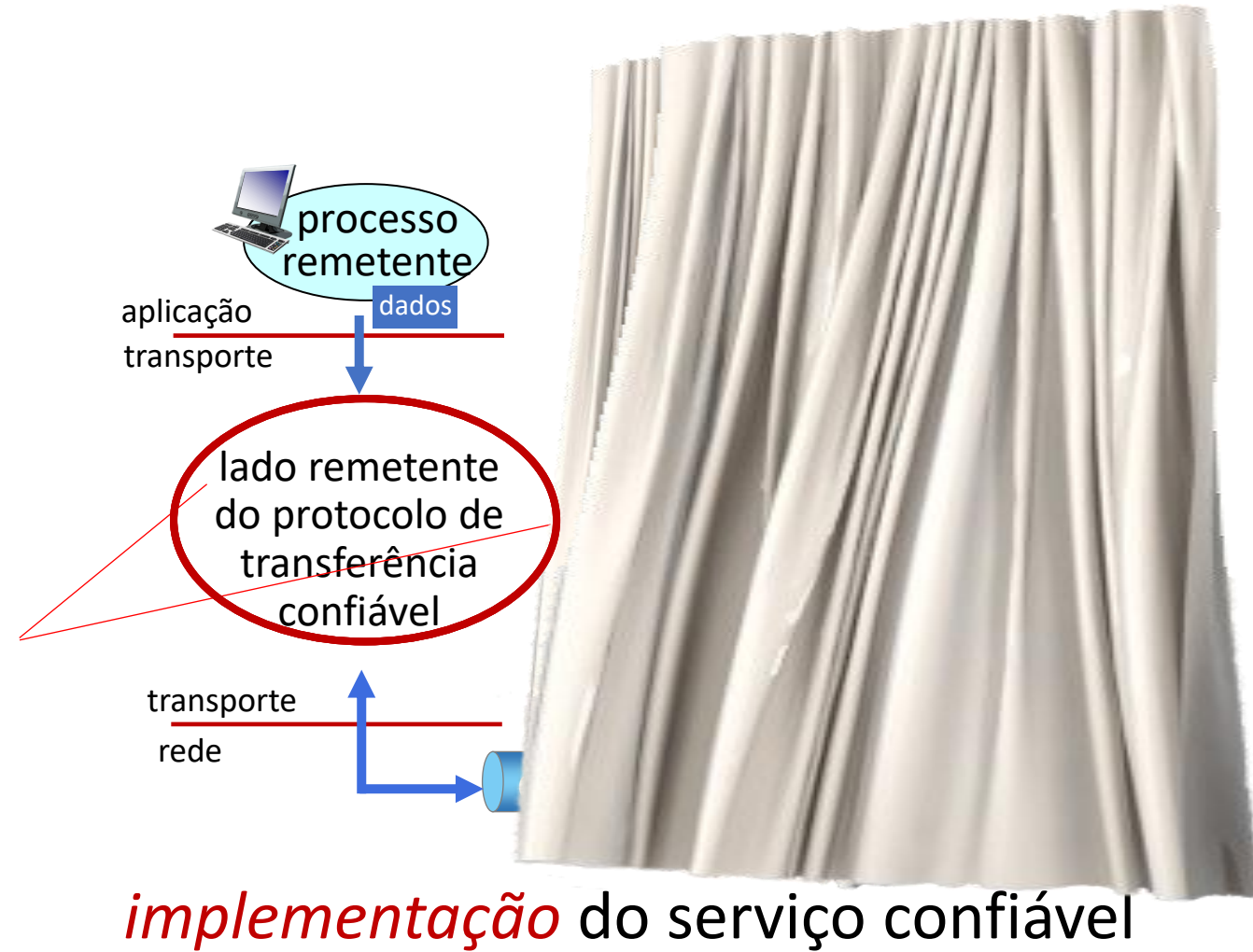


implementação do serviço confiável

Princípios de Transferência confiável de dados

Remetente, receptor *não* conhecem o “estado” um do outro, ex., a mensagem foi recebida?

- a não ser que seja informado através de uma mensagem



NAKs

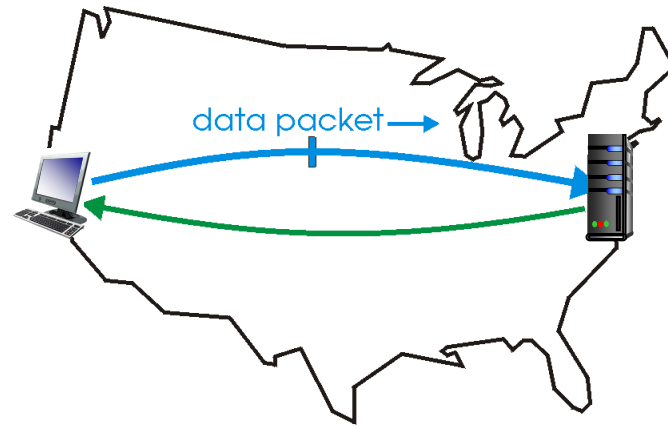
- ao invés de NAK, receptor envia ACK para último pacote recebido sem erro
 - receptor deve incluir *explicitamente* no. de seq do pacote reconhecido
- ACKs duplicados no transmissor resultam na mesma ação do NAK: *retransmissão do pacote atual*

Como veremos, TCP usa essa abordagem para não usar NAKs

Operação de protocolos com paralelismo

paralelismo (*pipelining*): transmissor envia vários pacotes em sequência, todos esperando para serem reconhecidos

- faixa de números de sequência deve ser aumentada
- armazenamento no transmissor e/ou no receptor

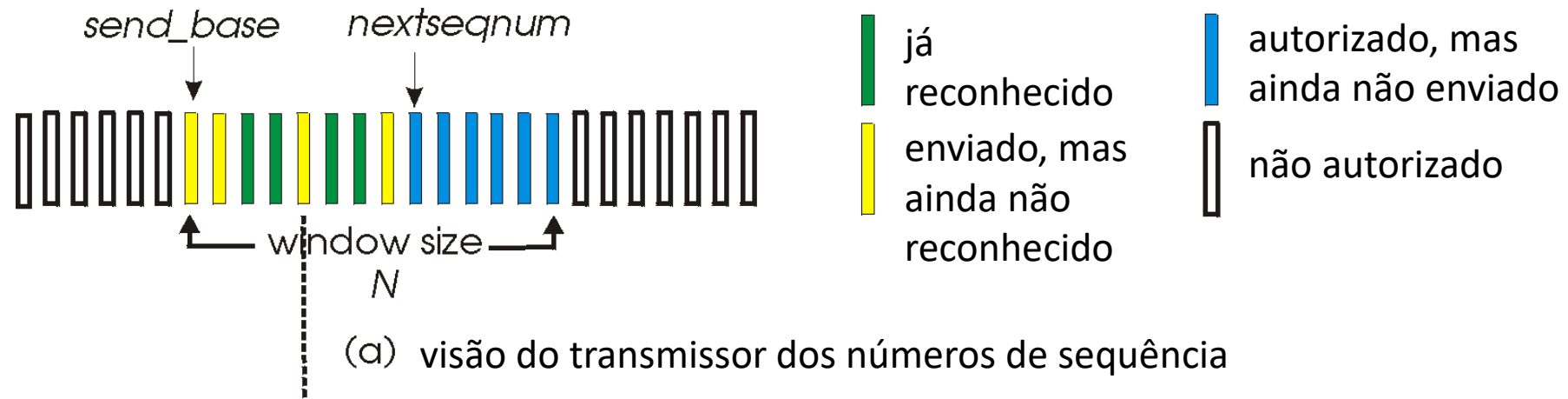


(a) a stop-and-wait protocol in operation

Repetição Seletiva: a abordagem

- *paralelismo*: múltiplos pacotes em trânsito
- *receptor reconhece individualmente* todos os pacotes recebidos corretamente
 - buferiza pacotes, caso necessário, para entrega em ordem à camada superior
- transmissor:
 - mantém (conceitualmente) um temporizador para cada pacote não reconhecido
 - *timeout*: retransmite apenas o pacote associado ao *timeout*
 - mantém (conceitualmente) “janela” incluindo *N* #s de seq. consecutivos
 - limita os pacotes em trânsito àqueles que cabem nessa janela

Retransmissão seletiva: janelas do transmissor e do receptor



Retransmissão seletiva: transmissor e receptor

transmissor

dados de cima:

- se próx. no. seq. disponível (n) na janela, envia pacote, liga temporizador (n)

timeout(n):

- reenvia pacote n , reinicia temporizador

ACK(n) em $[\text{sendbase}, \text{sendbase}+N]$:

- marca pacote n como recebido, desliga temporizador(n)
- se n for o menor pacote não reconhecido, avança a base da janela para o próx. no. de seq. não reconhecido

receptor

pacote n em $[\text{rcvbase}, \text{rcvbase}+N-1]$

- envia ACK(n)
- fora de ordem: armazena
- em ordem: entrega (tb. entrega pacotes armazenados em ordem), avança janela p/ próximo pacote ainda não recebido

pacote n em $[\text{rcvbase}-N, \text{rcvbase}-1]$

- ACK(n)

senão:

- ignora

Capítulo 3: roteiro

- Serviços da camada de transporte
- Multiplexação e demultiplexação
- Transporte sem conexões: UDP
- Princípios da transferência confiável
- **Transporte orientado a conexões: TCP**
 - estrutura do segmento
 - transferência confiável de dados
 - controle de fluxo
 - gerenciamento da conexão
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte

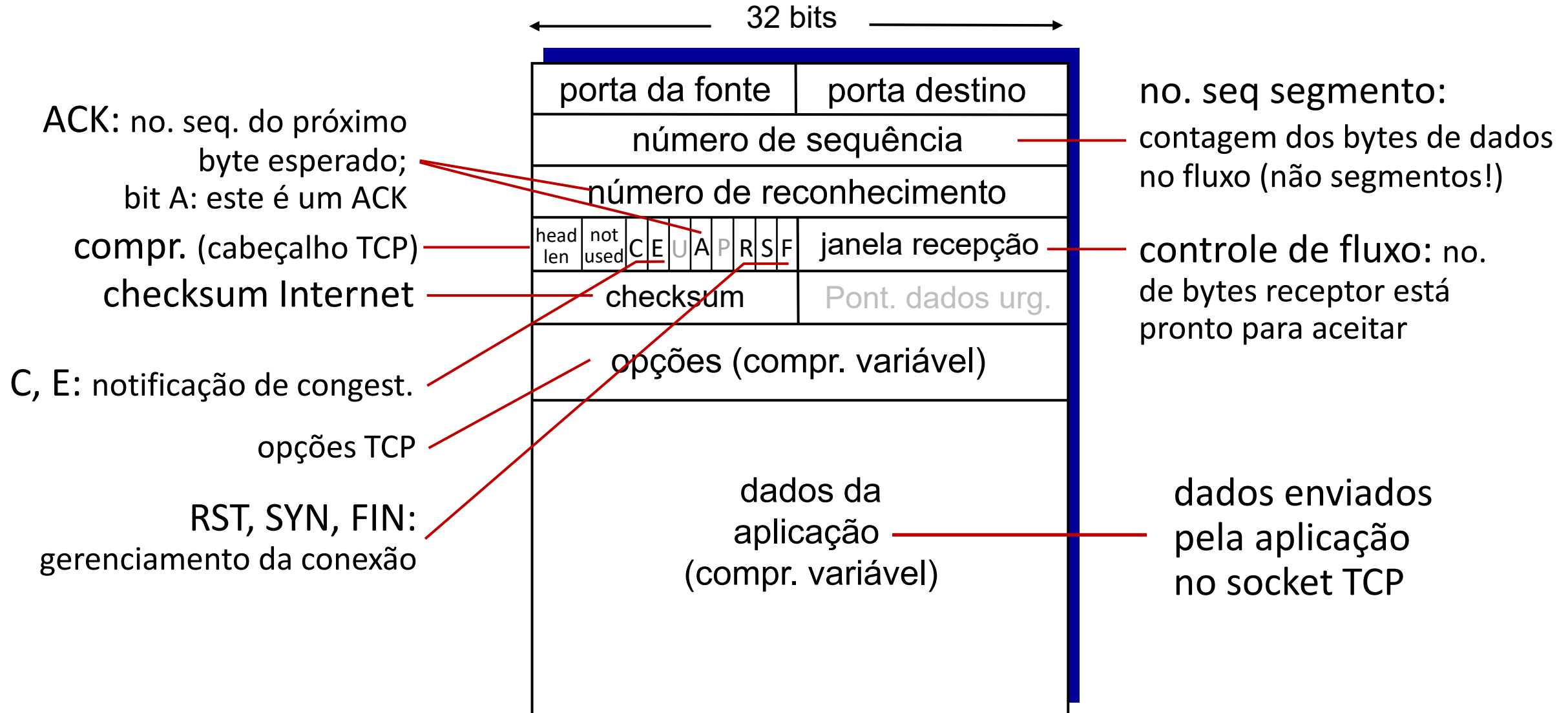


TCP: visão geral

RFCs: 793, 1122, 2018, 5681, 7323

- **ponto-a-ponto:**
 - um transmissor, um receptor
- **fluxo de bytes ordenados, confiável:**
 - sem “limites de msgs”
- **dados full duplex:**
 - fluxo bidirecional de dados na mesma conexão
 - MSS: tamanho máximo do segmento
- **ACKs cumulativos**
- **paralelismo:**
 - controle de congestionamento e de fluxo determinam tamanho da janela
- **orientado a conexões:**
 - *handshaking* (troca de mensagens de controle) inicializa estados do transmissor, receptor antes da troca de dados
- **fluxo controlado:**
 - transmissor não inundará o receptor

Estrutura do segmento TCP



TCP: números de sequência, ACKs

Números de sequência:

- “número”, dentro do fluxo de bytes, do primeiro byte de dados do segmento

Reconhecimentos (ACKs):

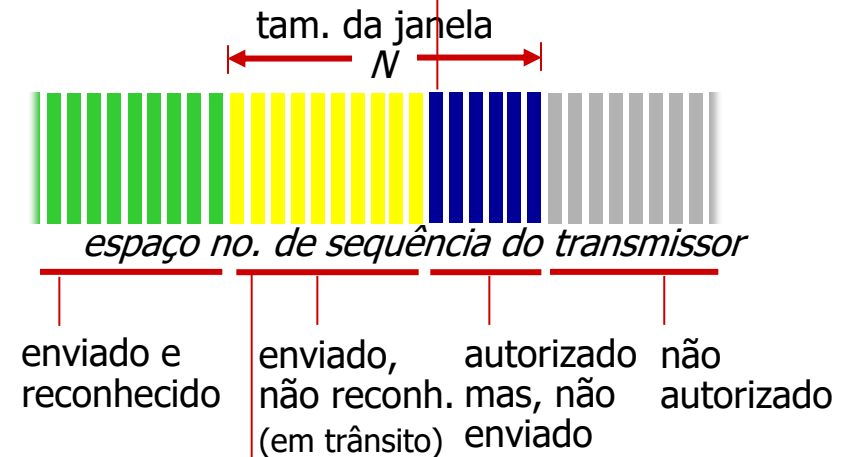
- no. de seq. do próx. byte esperado do outro lado
- ACK cumulativo

Q: como receptor trata segmentos fora da ordem?

- R: espec. do TCP omissa – deixado ao implementador

segmento de saída do transmissor

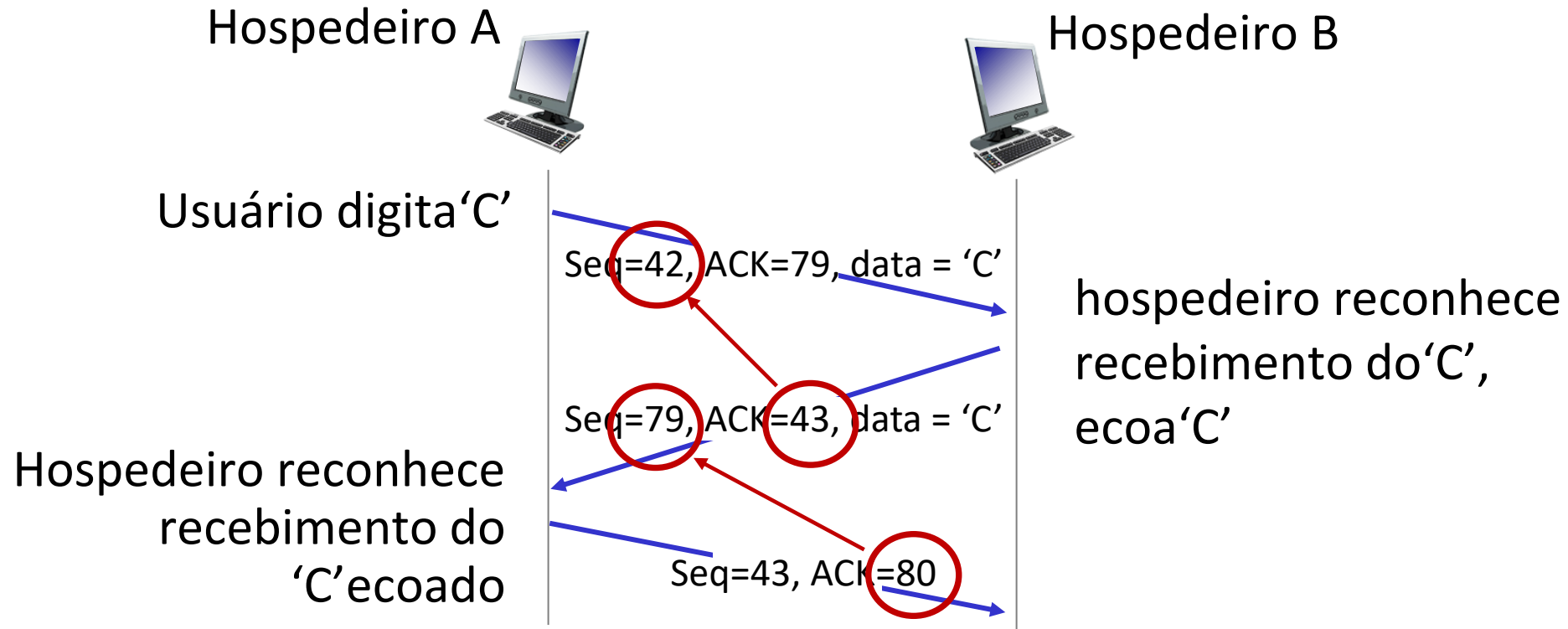
source port #	dest port #
número de sequência	
acknowledgement number	
	rwnd
checksum	urg pointer



segmento de saída do receptor

source port #	dest port #
sequence number	
número reconhecimento	
	A rwnd
checksum	urg pointer

TCP: números de sequência, ACKs



cenário telnet simples

TCP: tempo de ida e volta, temporizador

Q: como escolher o valor do temporizador TCP?

- maior que o RTT, mas o RTT varia!
- *muito curto*: temporização prematura, retransmissões desnecessárias
- *muito longo*: reação lenta à perda de segmentos

Q: como estimar o RTT?

- *SampleRTT*: tempo medido entre a transmissão do segmento e o recebimento do ACK correspondente
 - ignora retransmissões
- *SampleRTT* varia muito, é desejável um “amortecedor” para a estimativa do RTT
 - usa média de medições *recentes*, não apenas o último *SampleRTT* obtido.

Transmissor TCP (simplificado)

evento: dados recebidos da aplicação

- cria segmento com no. de sequência (nseq)
- nseq é o número de sequência do primeiro byte de dados do segmento
- liga o temporizador se já não estiver ligado
 - temporização do segmento mais antigo ainda não reconhecido
 - valor do temporizador: **TimeoutInterval**

evento: estouro do temporizador

- retransmite o segmento que causou o estouro do temporizador
- reinicia o temporizador

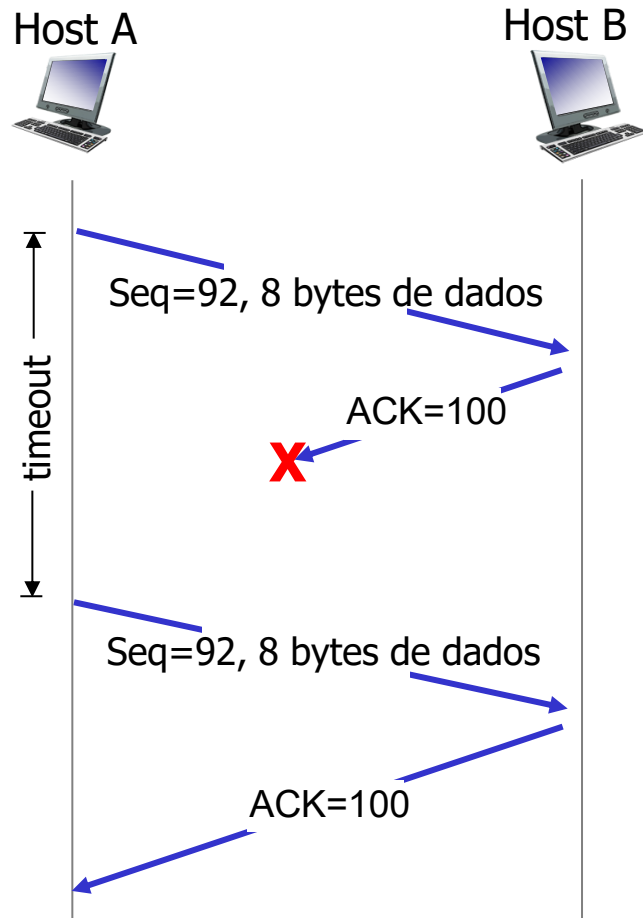
evento: recebimento de ACK

- se o ACK reconhece segmentos ainda não reconhecidos
 - atualizar informação sobre o que foi reconhecido
 - religa o temporizador se ainda houver segmentos pendentes (não reconhecidos),
 - caso contrário, desliga o temporizador.

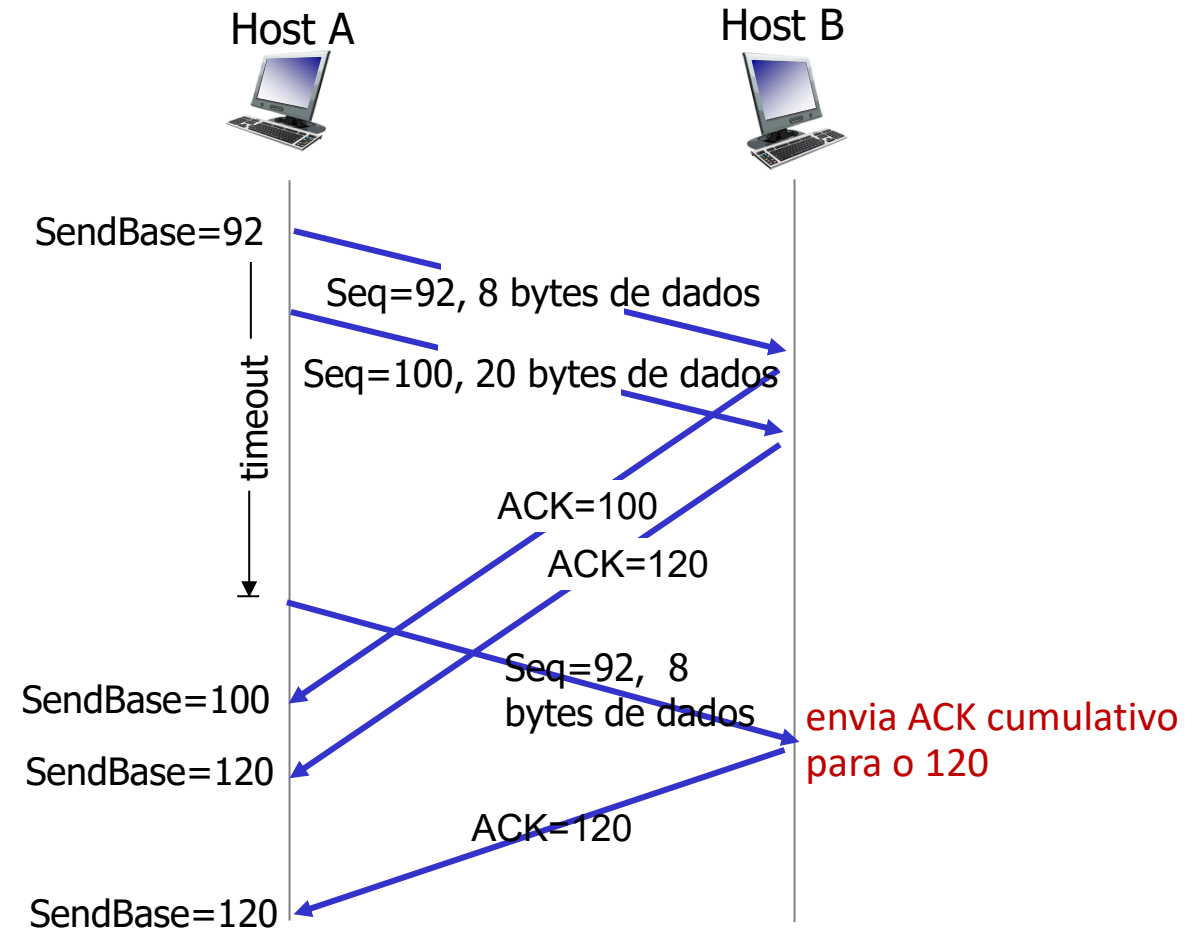
Receptor TCP: geração de ACKs [RFC 5681]



TCP: cenários de retransmissão

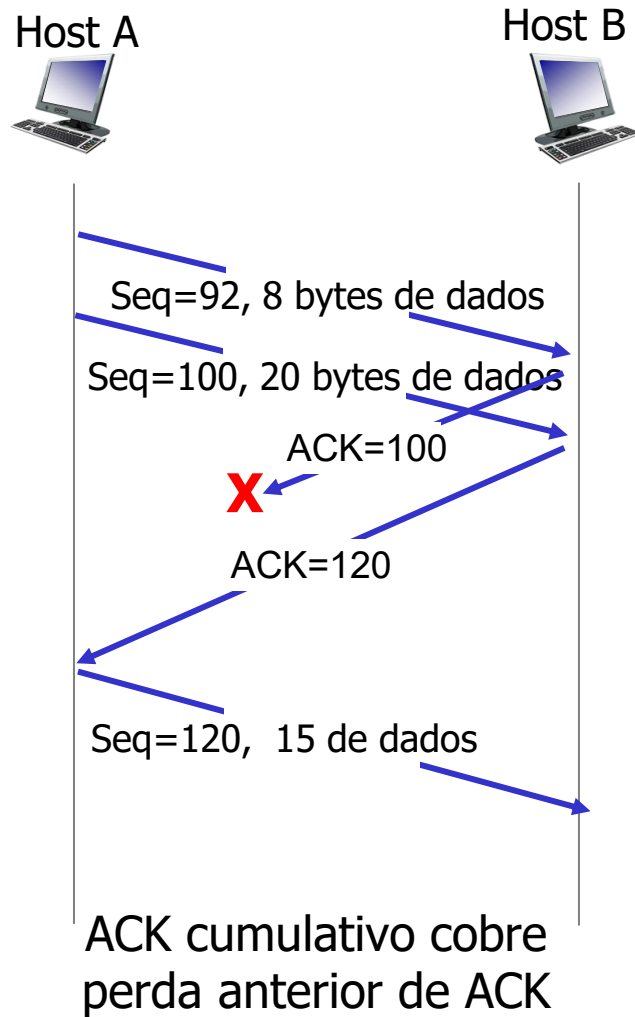


cenário com perda
do ACK



estouro prematuro
do temporizador

TCP: cenários de retransmissão



Retransmissão rápida do TCP

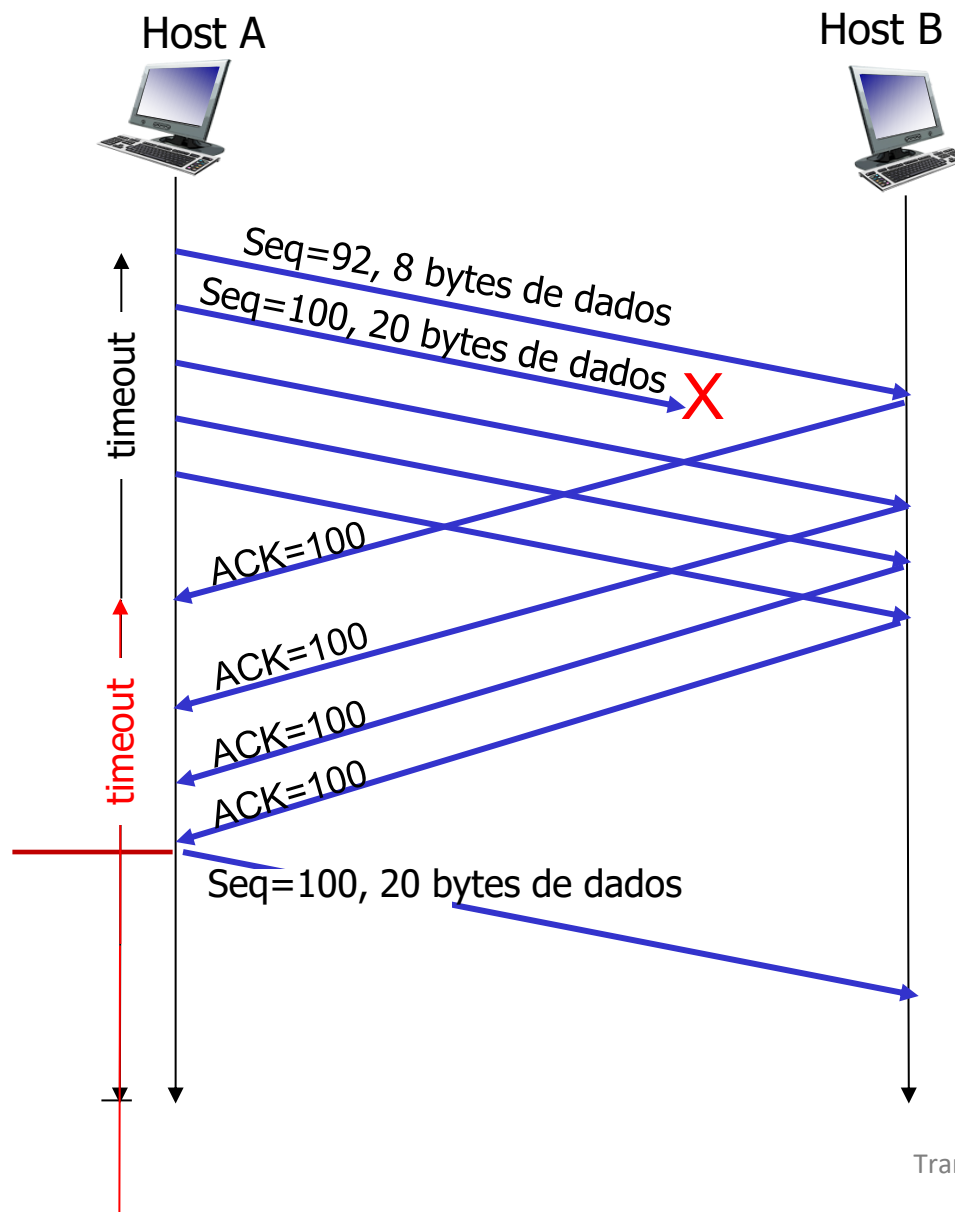
Retransmissão rápida do TCP

se o transmissor receber 3 ACKs para os mesmos dados (“três ACKs duplicados”), retransmite segmento não reconhecido com menor no. de seq.

- provavelmente o segmento não reconhecido se perdeu, não espera pelo temporizador



Recepção de três ACKs duplicados indica que 3 segmentos foram recebidos depois de um não recebido, provavelmente perdido. Então, retransmite!



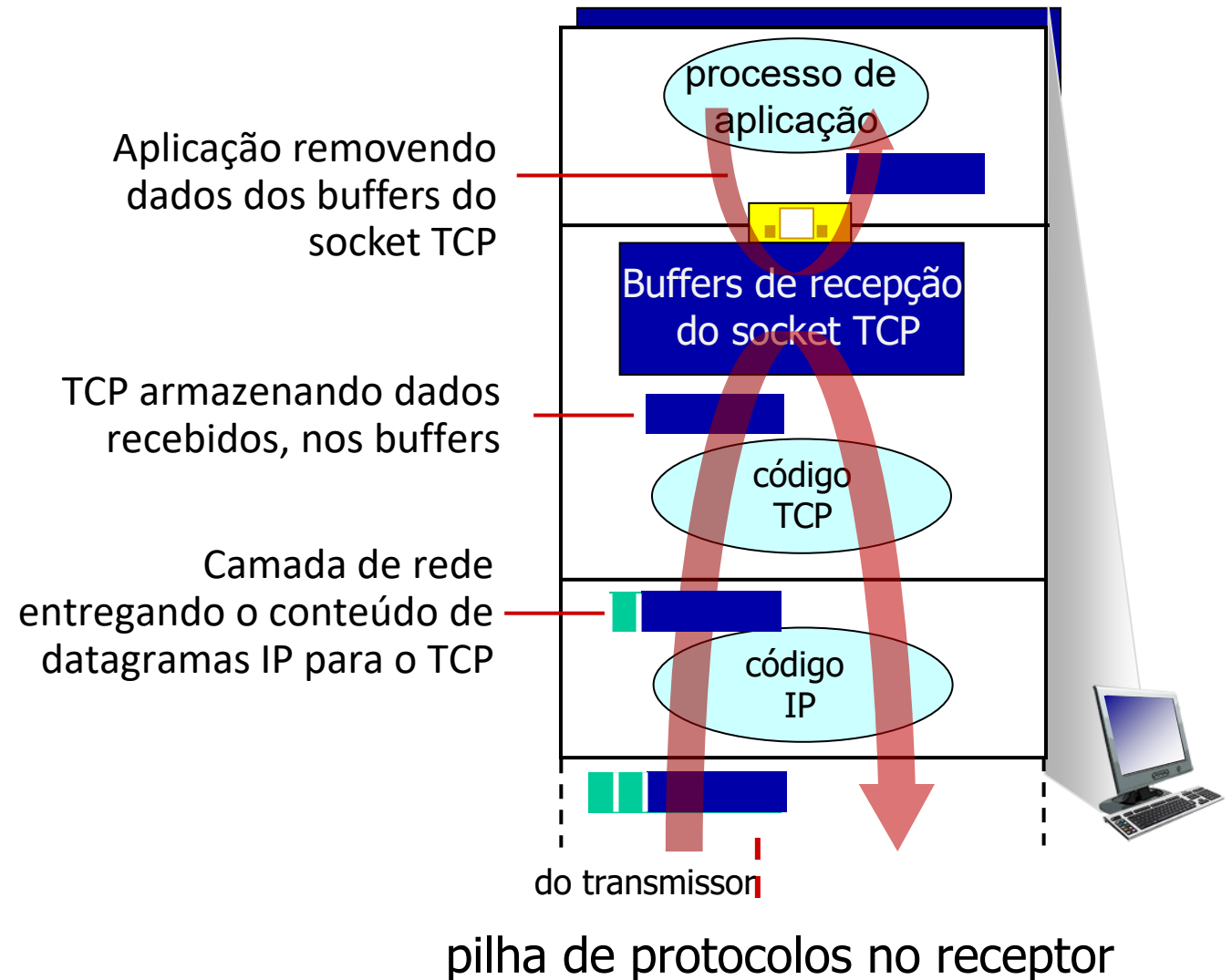
Capítulo 3: roteiro

- Serviços da camada de transporte
- Multiplexação e demultiplexação
- Transporte sem conexões: UDP
- Princípios da transferência confiável
- **Transporte orientado a conexões: TCP**
 - estrutura do segmento
 - transferência confiável de dados
 - controle de fluxo
 - gerenciamento da conexão
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte



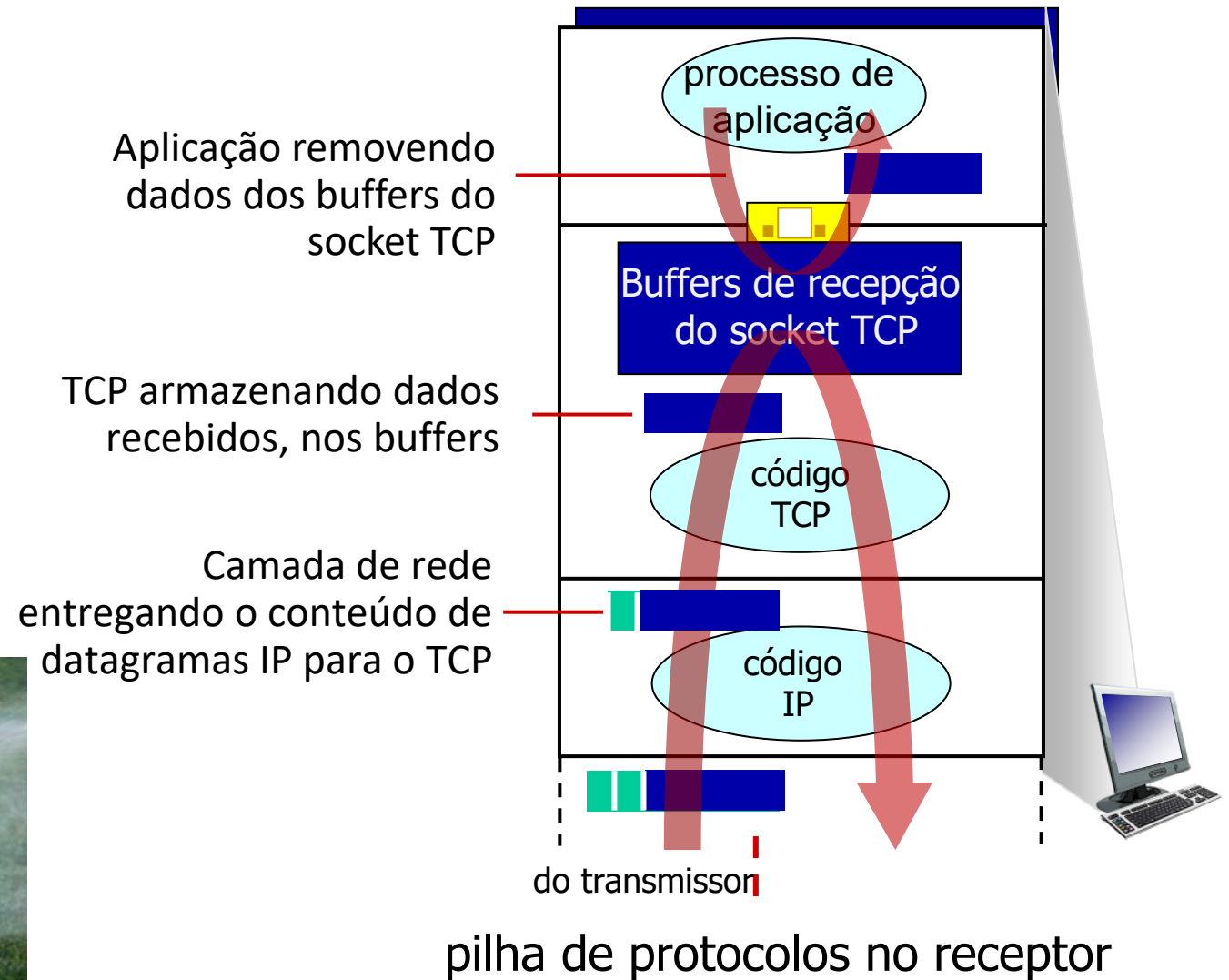
Controle de fluxo do TCP

Q: O que acontece se a camada de rede entrega dados mais rápido do que a camada de aplicação remove os dados dos buffers do socket?



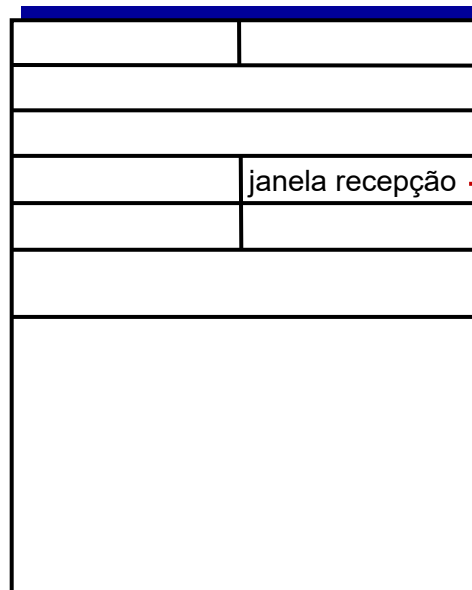
Controle de fluxo do TCP

Q: O que acontece se a camada de rede entrega dados mais rápido do que a camada de aplicação remove os dados dos buffers do socket?



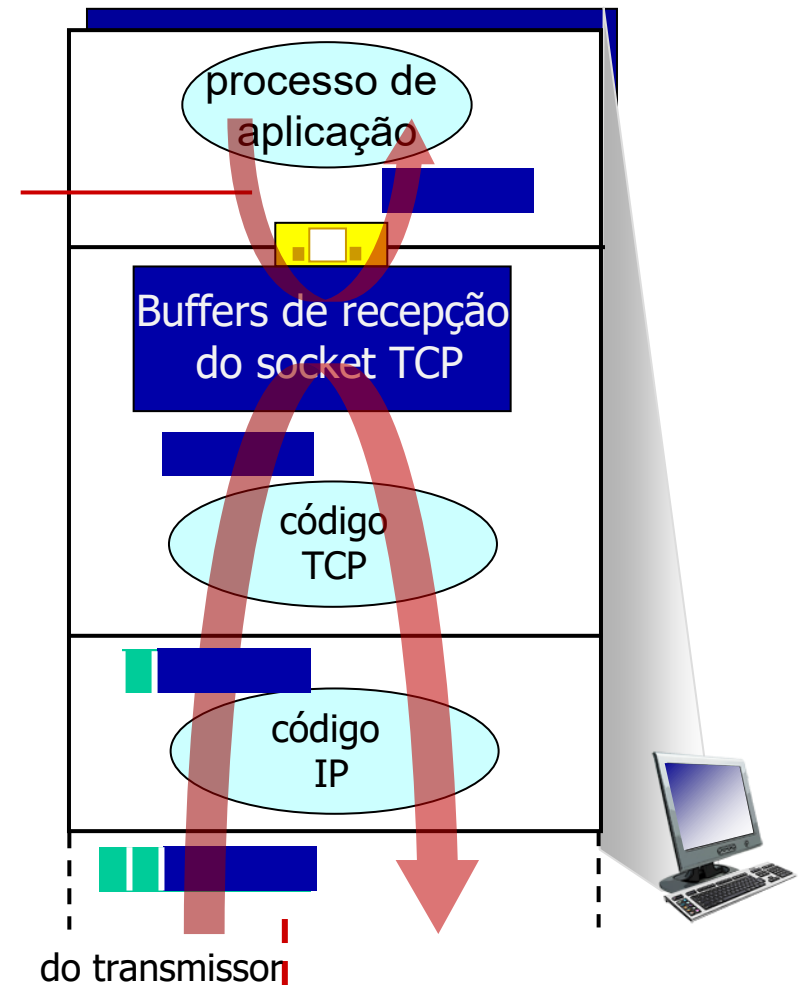
Controle de fluxo do TCP

Q: O que acontece se a camada de rede entrega dados mais rápido do que a camada de aplicação remove os dados dos buffers do socket?



controle de fluxo: no. bytes que o receptor está disposto a receber

Aplicação removendo dados dos buffers do socket TCP



pilha de protocolos no receptor

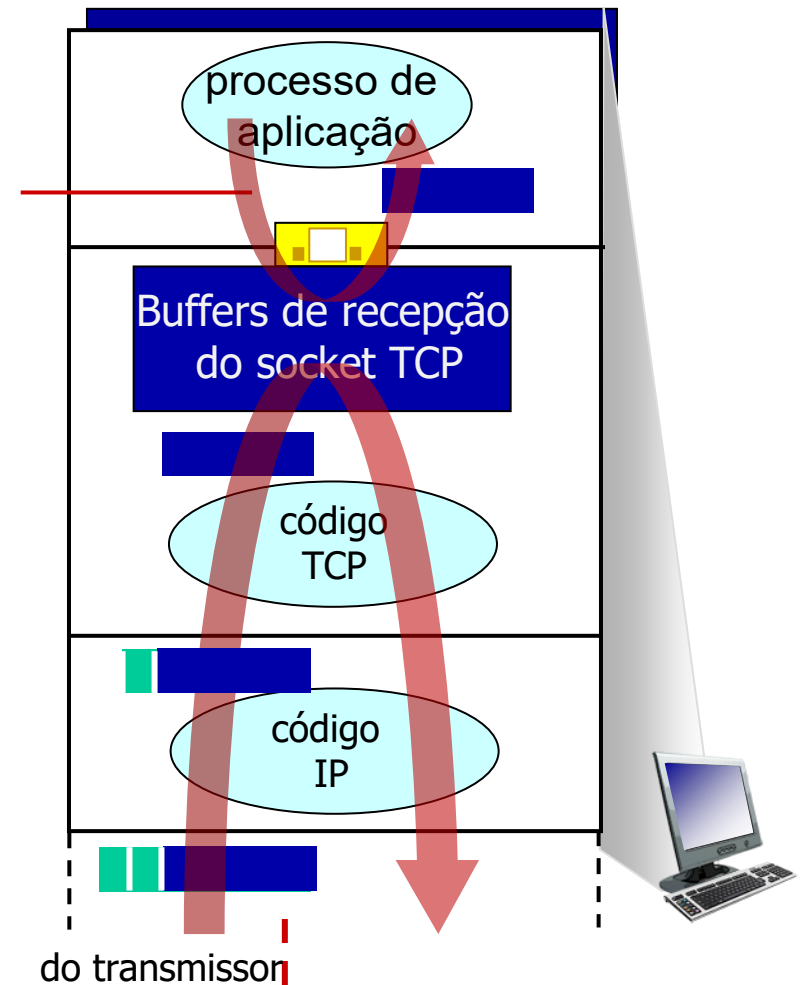
Controle de fluxo do TCP

Q: O que acontece se a camada de rede entrega dados mais rápido do que a camada de aplicação remove os dados dos buffers do socket?

controle de fluxo

receptor controla o transmissor, de modo que o transmissor não inunde o buffer do receptor, transmitindo muita coisa, muito rápido.

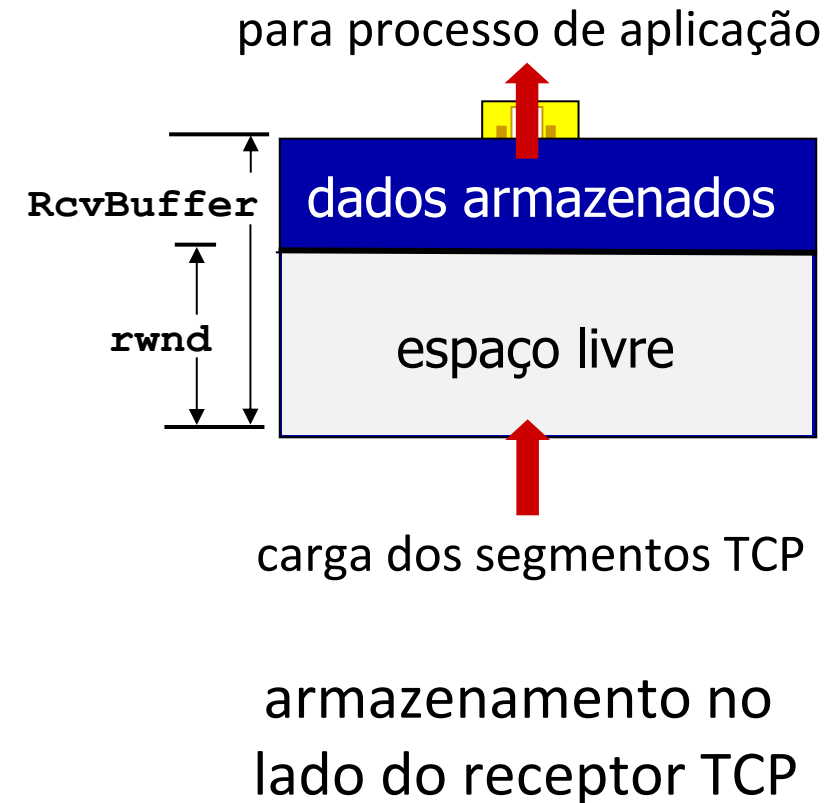
Aplicação removendo dados dos buffers do socket TCP



pilha de protocolos no receptor

Controle de fluxo do TCP

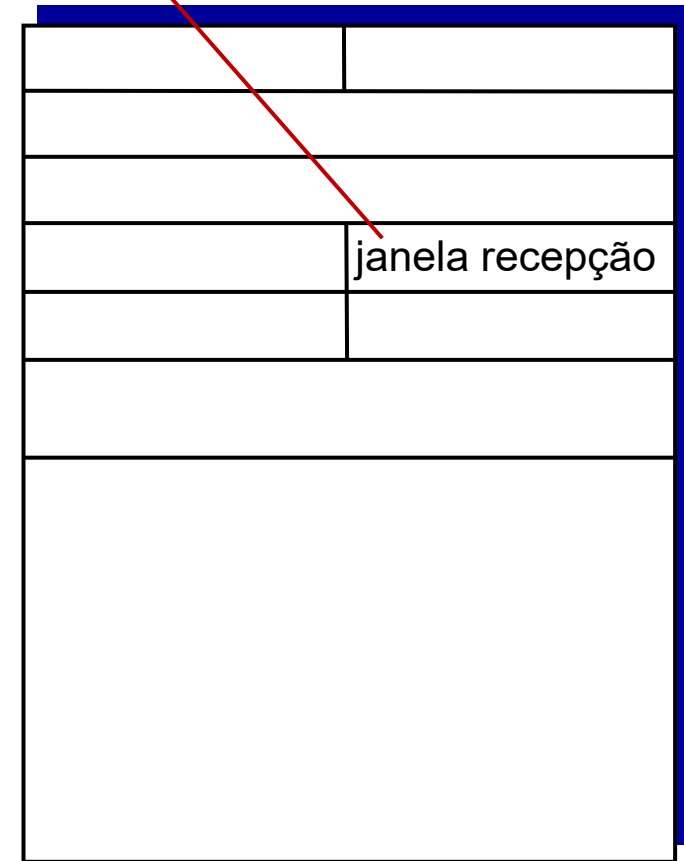
- O receptor TCP “anuncia” o espaço livre do buffer no campo **rwnd** (janela de recepção) no cabeçalho TCP
 - **RcvBuffer** tamanho configurado nas opções de socket (valor típico *default* é de 4096 bytes)
 - muitos sistemas operacionais ajustam automaticamente o **RcvBuffer**
- o transmissor limita a quantidade os dados não reconhecidos ao tamanho do **rwnd** recebido
- garante que o buffer do receptor não transbordará



Controle de fluxo do TCP

- O receptor TCP “anuncia” o espaço livre do buffer no campo **rwnd** (janela recepção) no cabeçalho TCP
 - **RcvBuffer** tamanho configurado nas opções de socket (valor típico *default* é de 4096 bytes)
 - muitos sistemas operacionais ajustam automaticamente o **RcvBuffer**
- o transmissor limita a quantidade os dados não reconhecidos ao tamanho do **rwnd** recebido
- garante que o buffer do receptor não transbordará

controle de fluxo: no. bytes que o receptor está disposto a receber

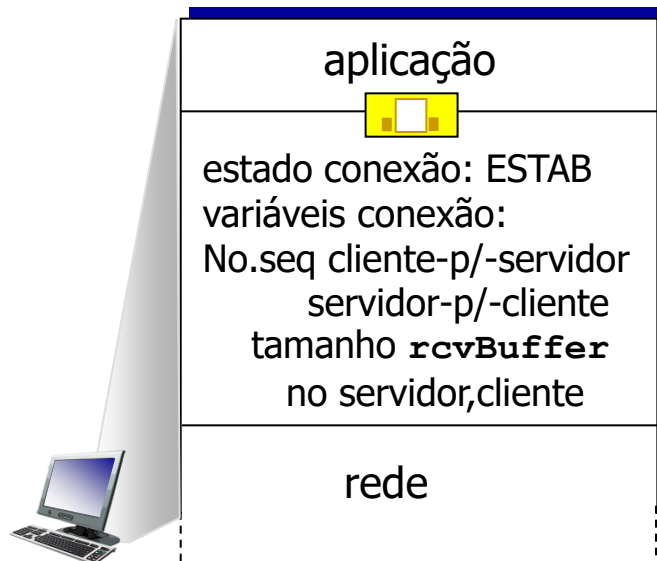


formato do segmento TCP

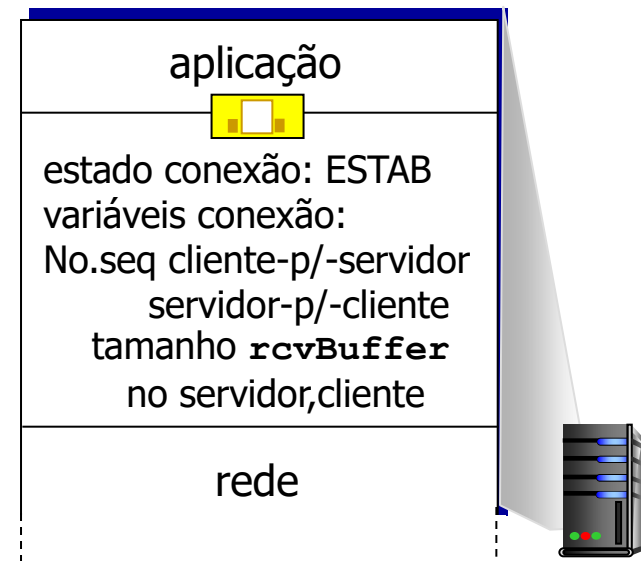
TCP: gerenciamento de conexões

antes de trocar dados, transmissor e receptor TCP dialogam:

- concordam em estabelecer uma conexão (cada um sabendo que o outro quer estabelecer a conexão)
- concordam com os parâmetros da conexão (ex., nos. seq. iniciais)



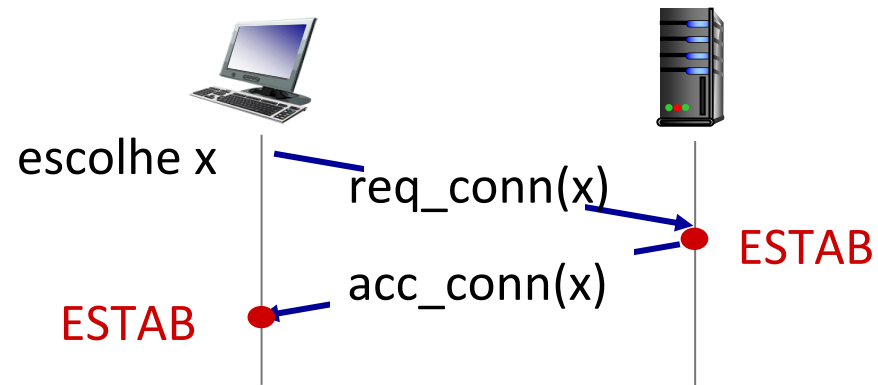
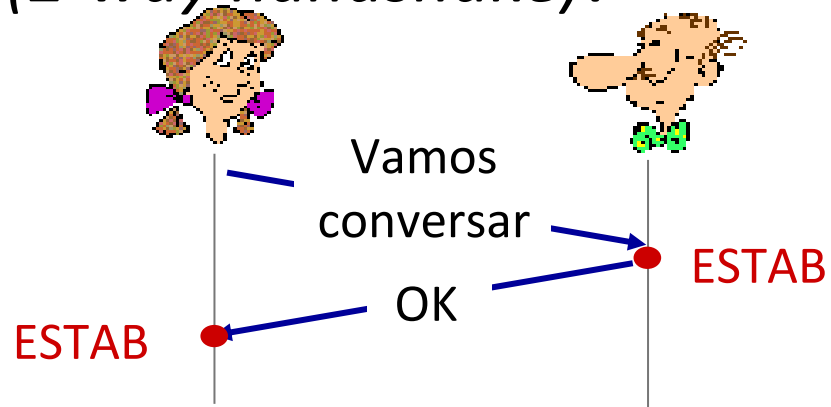
```
Socket clientSocket =  
    newSocket("hostname", "port number");
```



```
Socket connectionSocket =  
    welcomeSocket.accept();
```


Concordando em estabelecer uma conexão

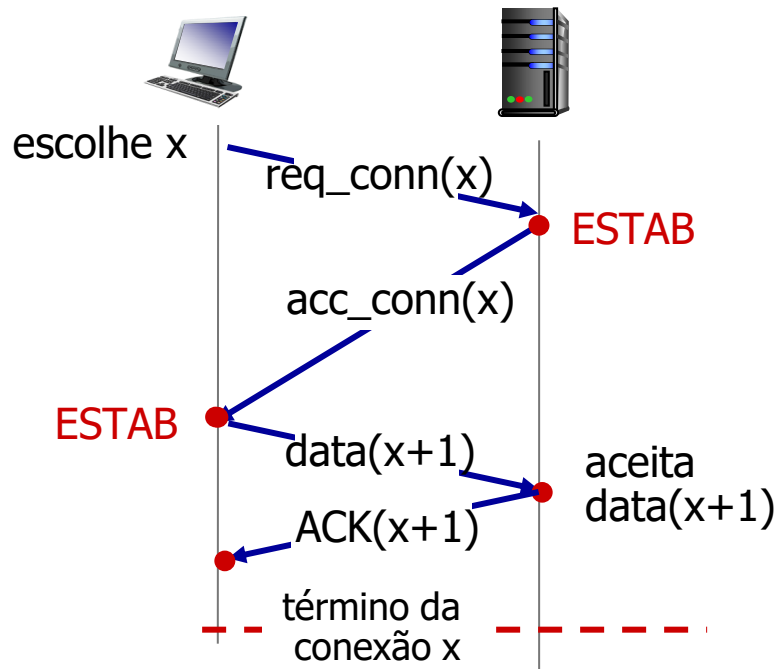
Apresentação de 2 vias
(2-way handshake):



Q: a apresentação em duas vias sempre funciona em redes?

- atrasos variáveis
- mensagens retransmitidas (ex. req_conn(x)) devido à perda de mensagem
- reordenação de mensagens
- não consegue “ver” o outro lado

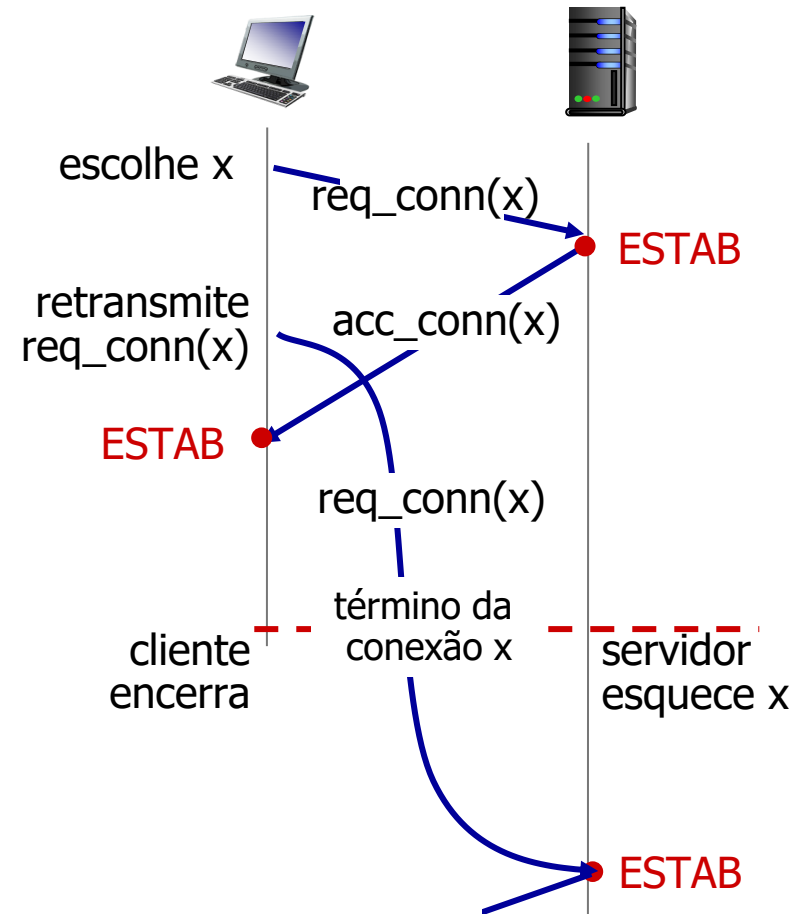
Cenários de apresentação de 2 vias



Sem problemas!

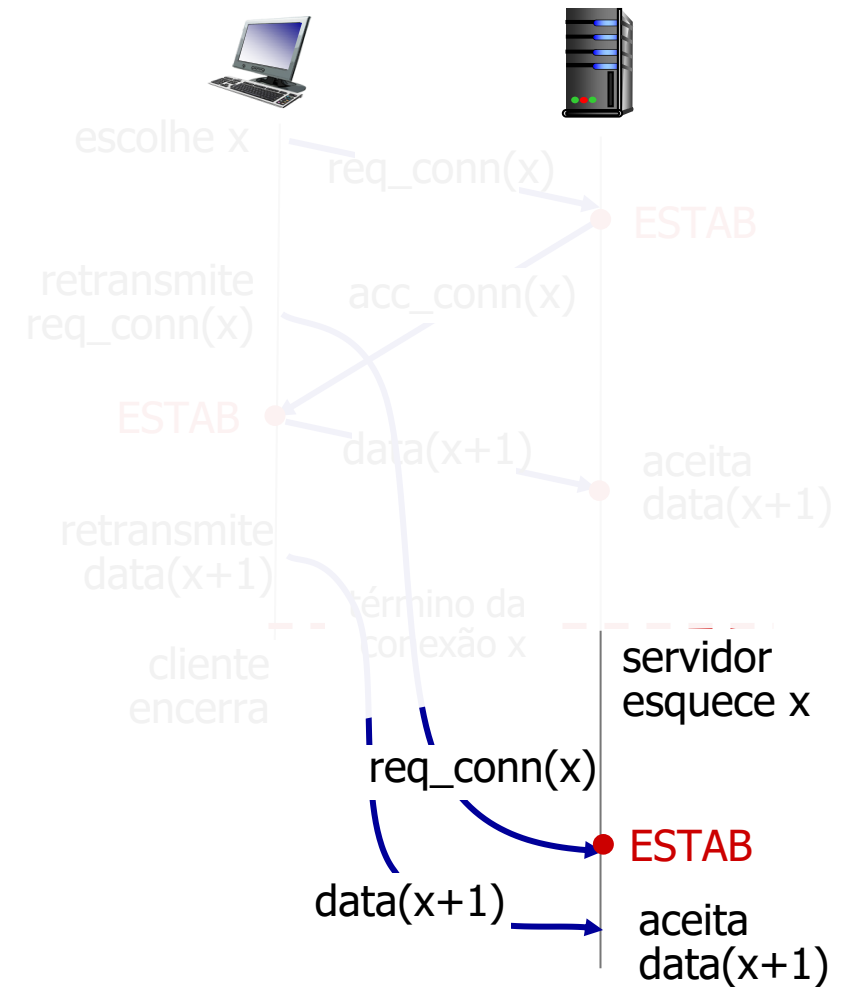


Cenários de apresentação de 2 vias



Problema: conexão aberta pela metade! (sem cliente)

Cenários de apresentação de 2 vias



❌ Problema: aceita dados duplicados!

Apresentação de três vias do TCP

Estado do servidor

```
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

Estado do cliente

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB

escolhe no. seq inicial, x
envia msg TCP SYN

SYNACK(x) recebido
indica servidor ativo;
envia ACK para SYNACK;
este segmento pode conter
dados do cliente para servidor



SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

ACKbit=1, ACKnum=y+1

escolhe no. seq inicial, y
envia msg TCP SYNACK,
reconhecendo o SYN

ACK(y) recebido
indica cliente ativo

LISTEN

SYN RCVD

ESTAB

Um protocolo humano de três vias



Encerrando uma conexão TCP

- seja o cliente que o servidor fecham cada um o seu lado da conexão
 - enviam segmento TCP com bit FIN = 1
- respondem ao FIN recebido com um ACK
 - ao receber um FIN, ACK pode ser combinado com o próprio FIN
- lida com trocas de FIN simultâneos

Capítulo 3: roteiro

- Serviços da camada de transporte
- Multiplexação e demultiplexação
- Transporte sem conexões: UDP
- Princípios da transferência confiável
- Transporte orientado a conexões: TCP
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte



Princípios de Controle de Congestionamento

Congestionamento:

- informalmente: “muitas fontes enviando dados acima da capacidade da *rede* de tratá-los”
- sintomas:
 - longos atrasos (enfileiramento nos buffers dos roteadores)
 - perda de pacotes (saturação de buffers nos roteadores)
- diferente de controle de fluxo!
- um dos 10 maiores problemas!



Controle de congestionamento:
demasiados transmissores,
enviando muito rápido

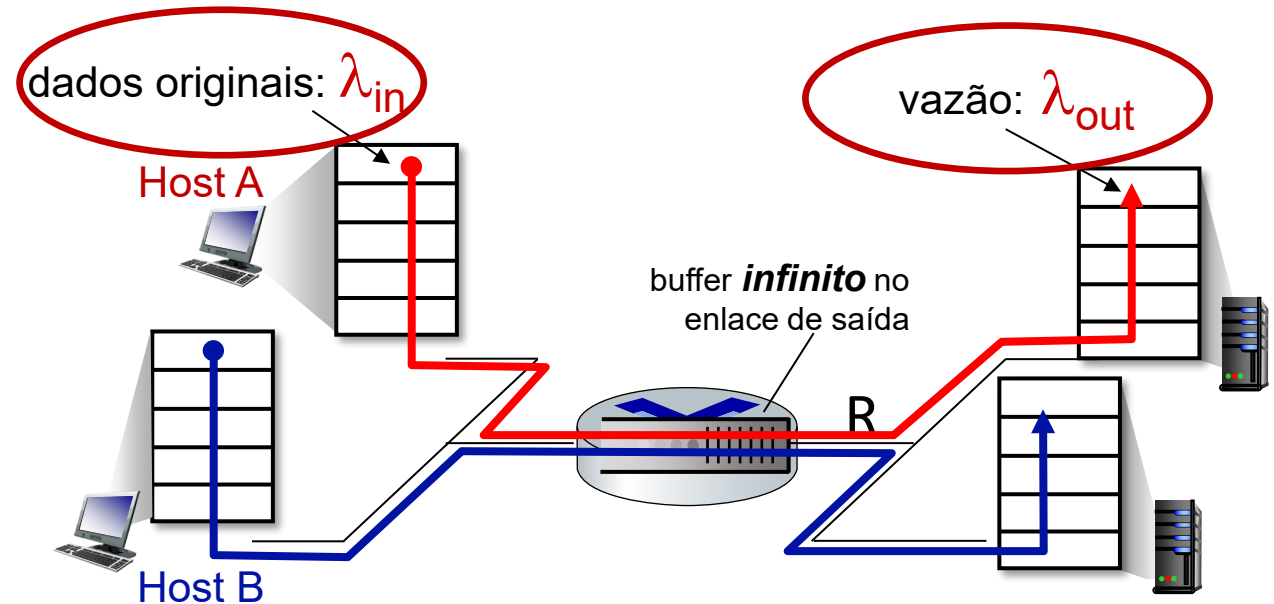


controle de fluxo: um
transmissor muito rápido
para um receptor

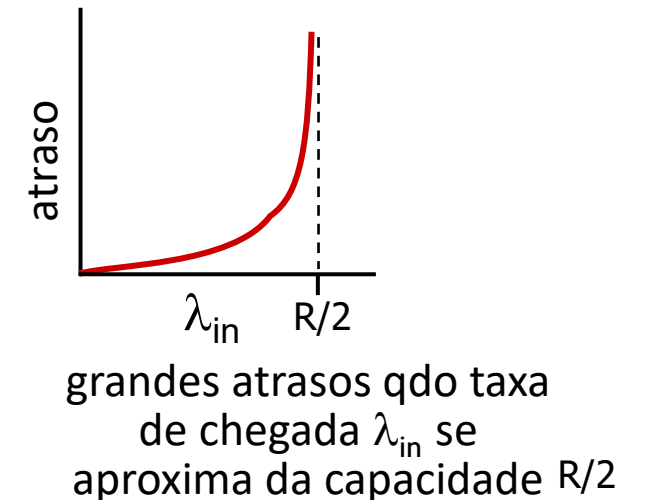
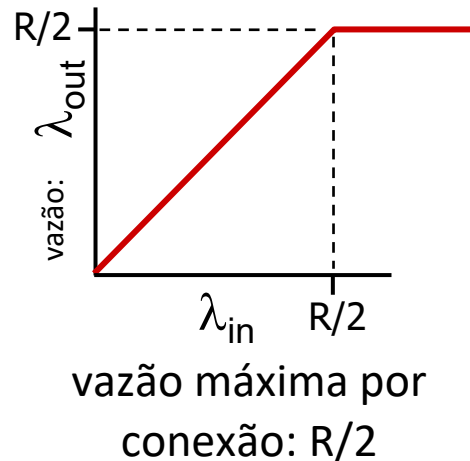
Causas/custos de congestionamento: cenário 1

Cenário mais simples:

- um roteador, buffers infinitos
- capacidade do enlace de saída: R
- dois fluxos
- sem retransmissões

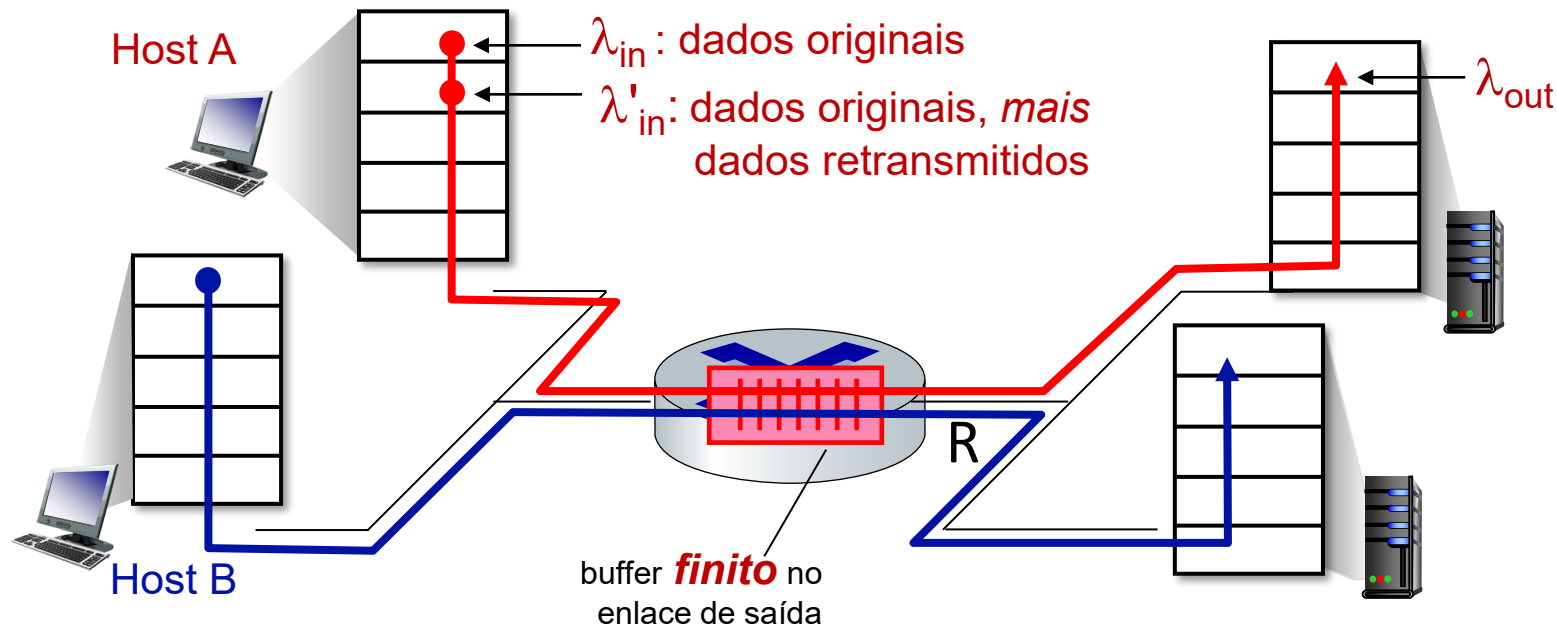


Q: O que acontece quando a taxa de chegada λ_{in} atinge $R/2$?



Causas/custos de congestionamento: cenário 2

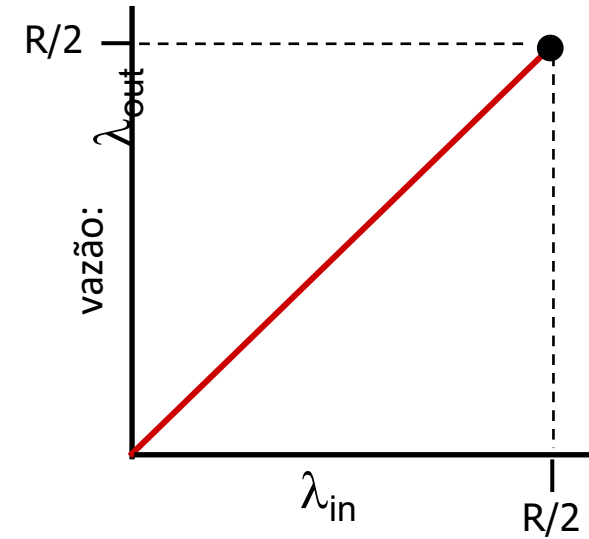
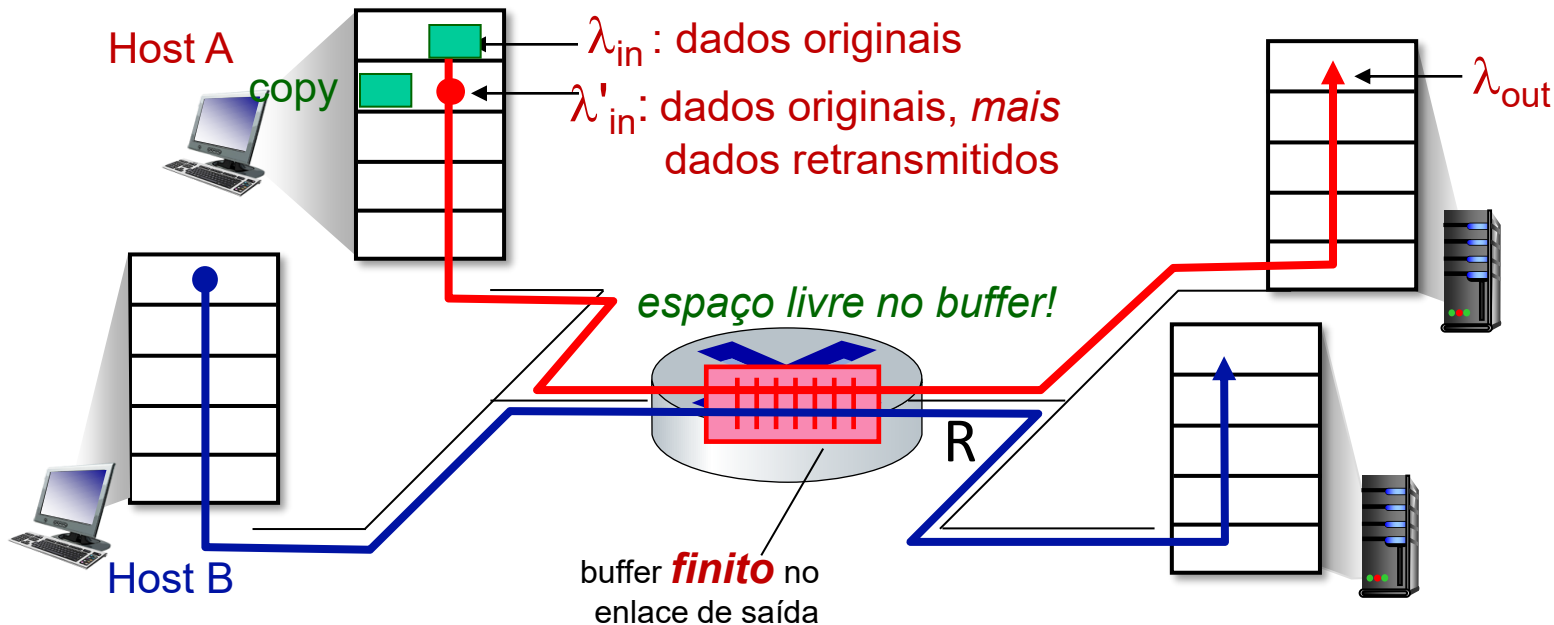
- um roteador, buffers *finitos*
- transmissor retransmite pacotes perdidos, estouro do temporizador
 - entrada camada de aplicação = saída da camada de aplicação: $\lambda_{in} = \lambda_{out}$
 - entrada camada de transporte inclui retransmissões: $\lambda'_{in} \geq \lambda_{in}$



Causas/custos de congestionamento: cenário 2

Idealização: conhecimento perfeito

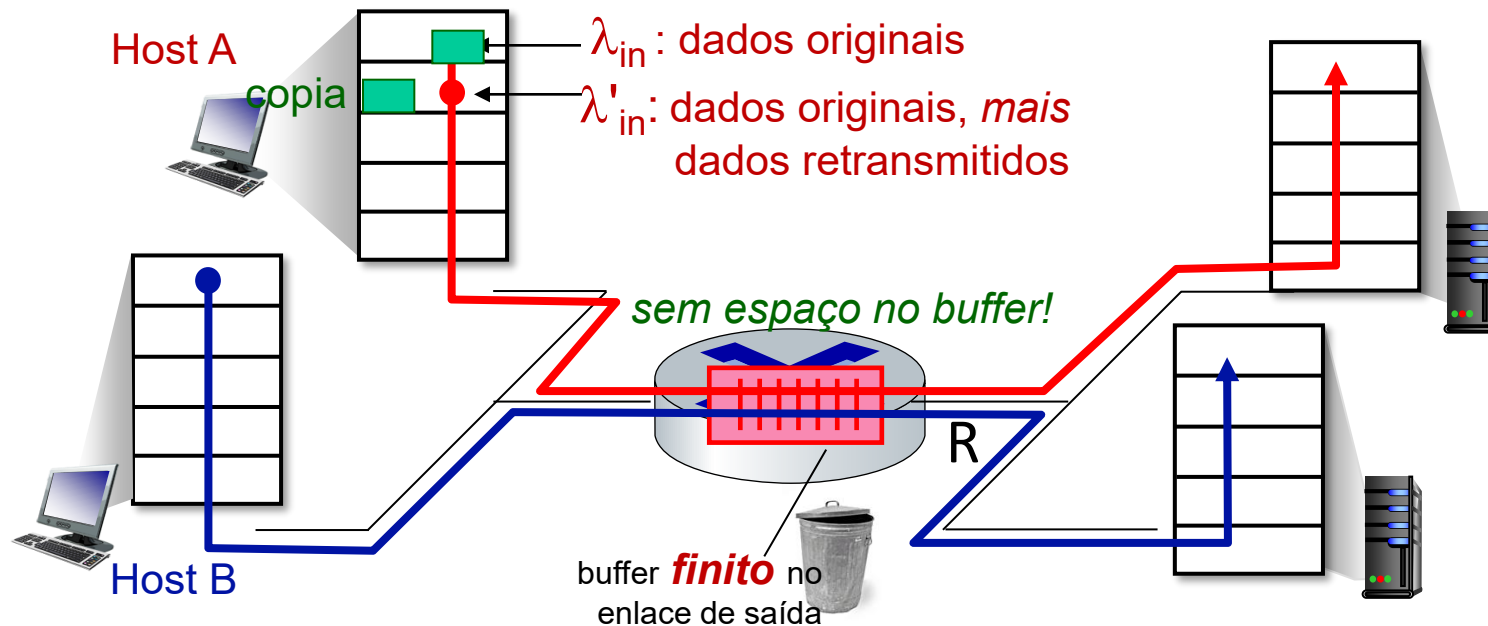
- transmissor envia apenas quando houver buffer disponível no roteador



Causas/custos de congestionamento: cenário 2

Idealização: *algum* conhecimento perfeito

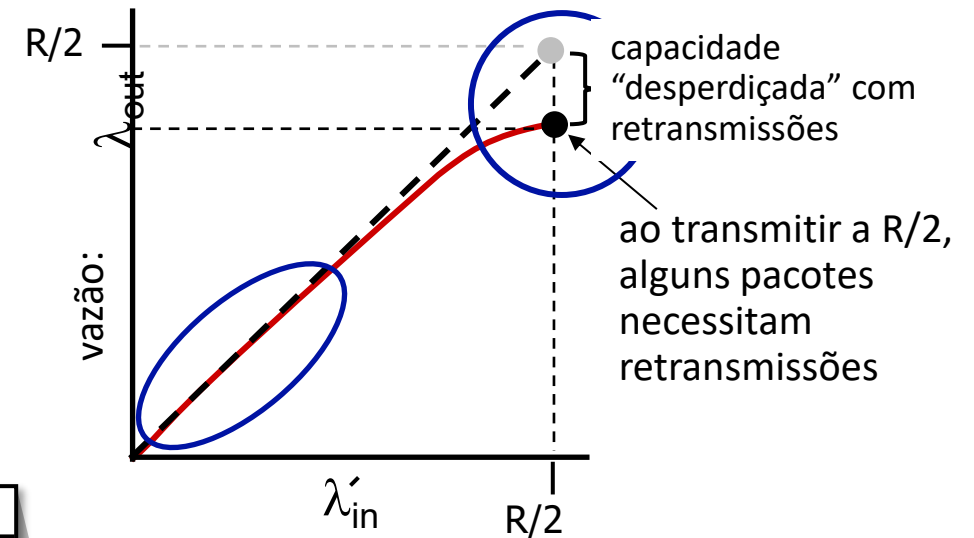
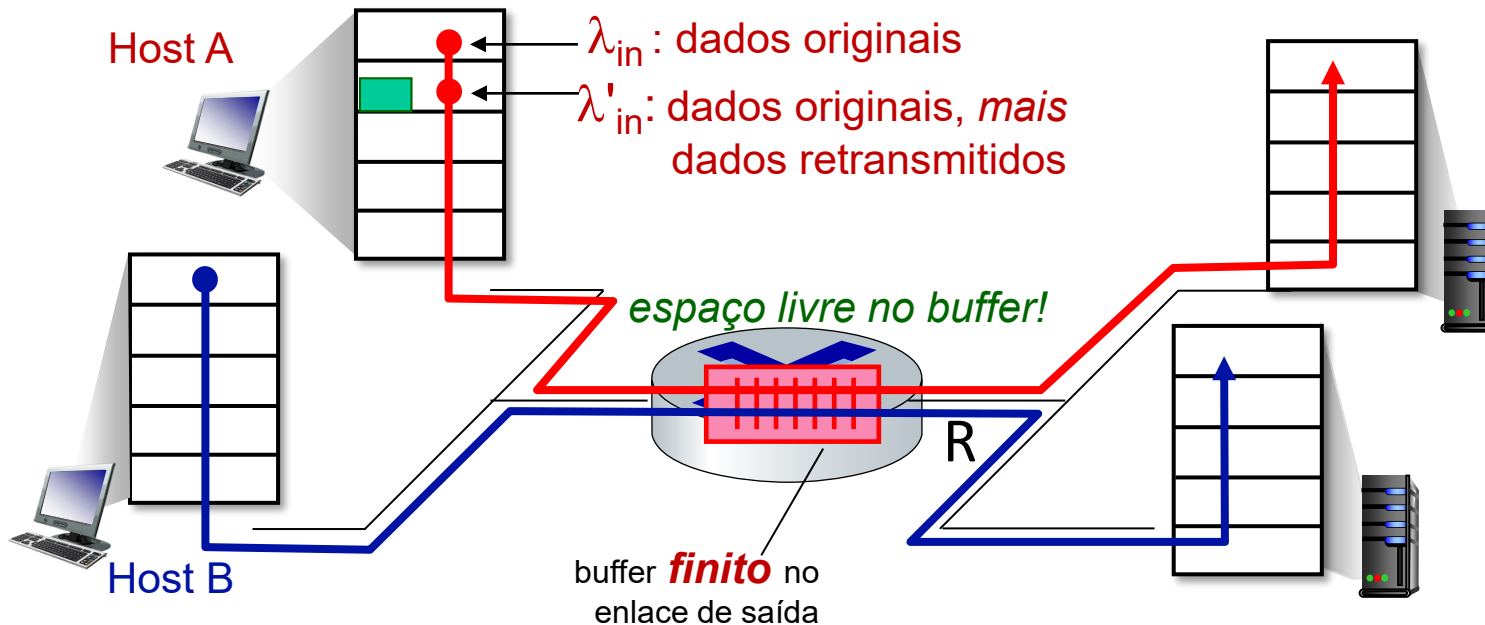
- pacotes podem ser perdidos (descartados no roteador) devido a buffers cheios
- transmissor sabe quando pacote foi descartado: retransmite apenas se o pacote *sabidamente* se perdeu



Causas/custos de congestionamento: cenário 2

Idealização: *algum* conhecimento perfeito

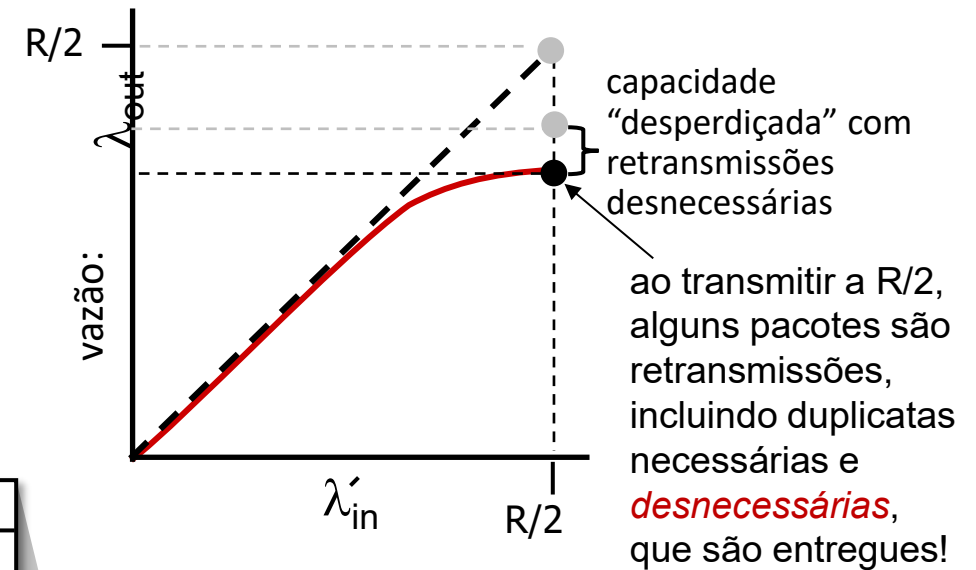
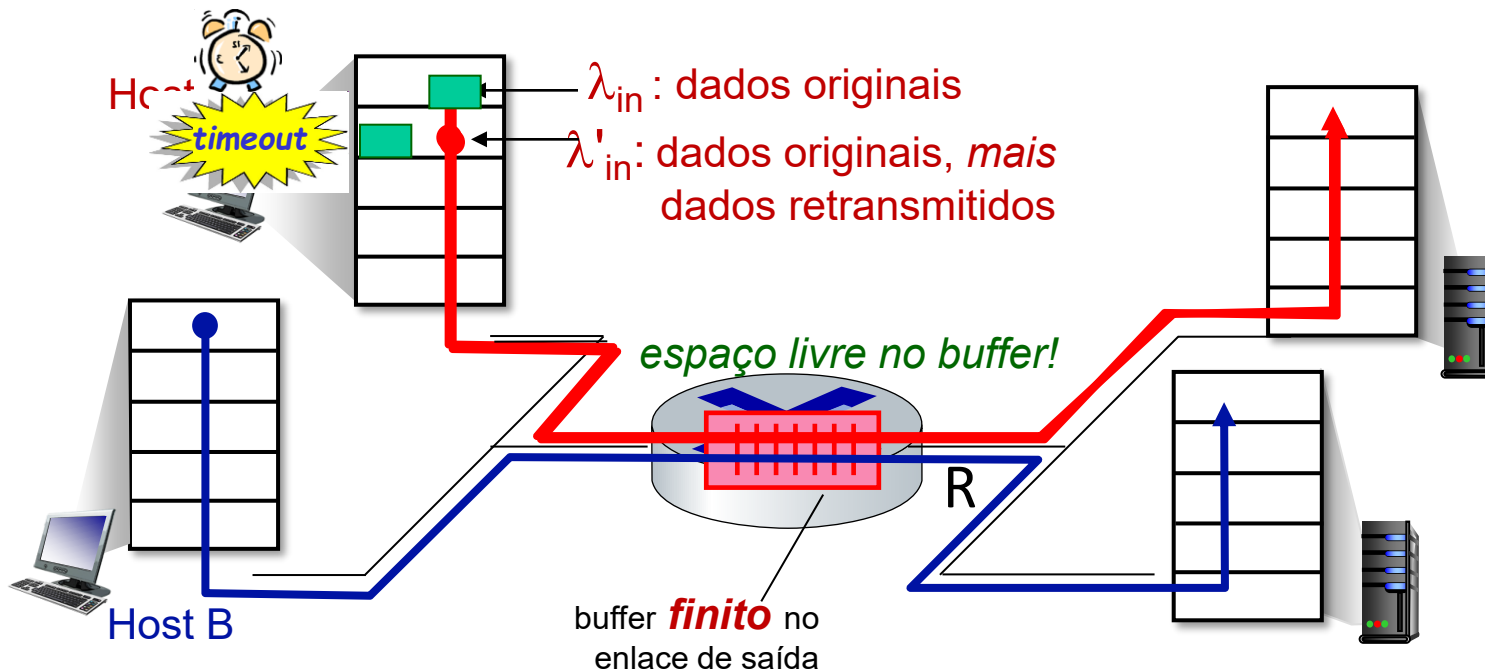
- pacotes podem ser perdidos (descartados no roteador) devido a buffers cheios
- transmissor sabe quando pacote foi descartado: retransmite apenas se o pacote *sabidamente* se perdeu



Causas/custos de congestionamento: cenário 2

Cenário realista: *duplicatas desnecessárias*

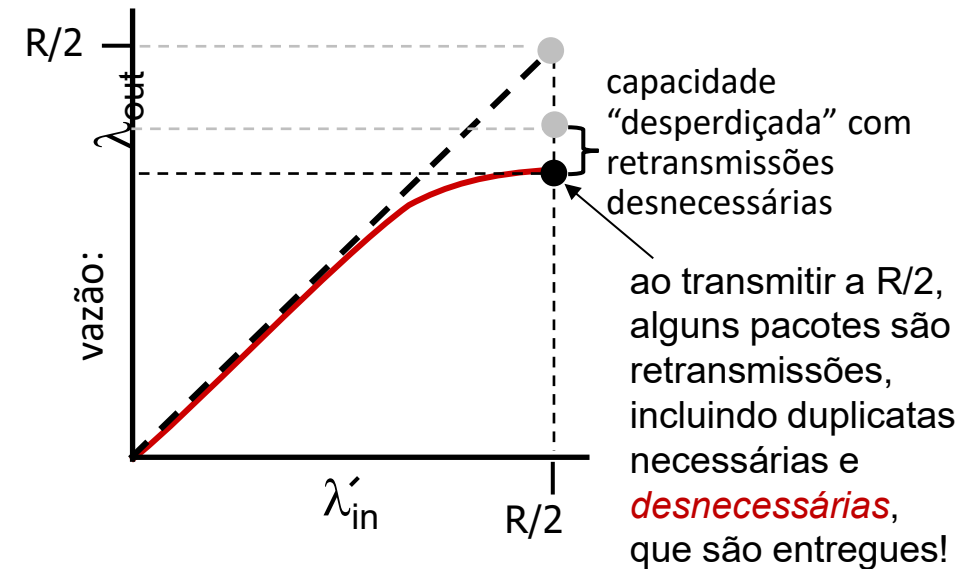
- pacotes podem ser perdidos, descartados no roteador, buffers cheios – requer retransmissões
- mas, temporizador pode estourar prematuramente enviando *duas* cópias, *ambas* entregues



Causas/custos de congestionamento: cenário 2

Cenário realista: *duplicatas desnecessárias*

- pacotes podem ser perdidos, descartados no roteador, buffers cheios – requer retransmissões
- mas, temporizador pode estourar prematuramente enviando *duas* cópias, *ambas* entregues



“custos” do congestionamento:

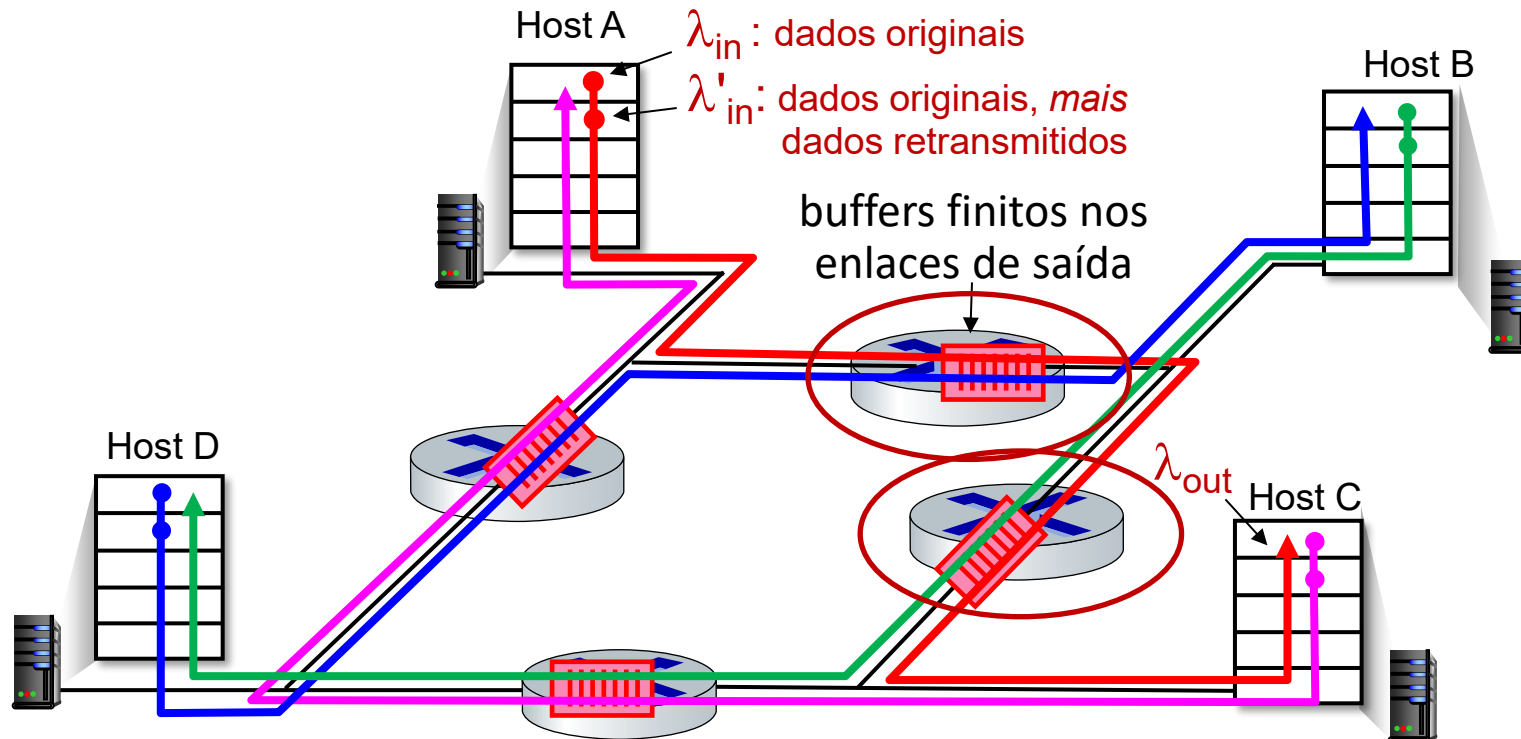
- mais trabalho (retransmissões) para uma dada vazão no receptor
- retransmissões desnecessárias: enlace transporta múltiplas cópias do pacote
 - diminuindo a vazão máxima alcançável

Causas/custos de congestionamento: cenário 3

- *quatro* transmissores
- caminhos *multi-enlaces*
- temporização/retransmissão

Q: o que ocorre à medida que λ_{in} e λ'_{in} crescem?

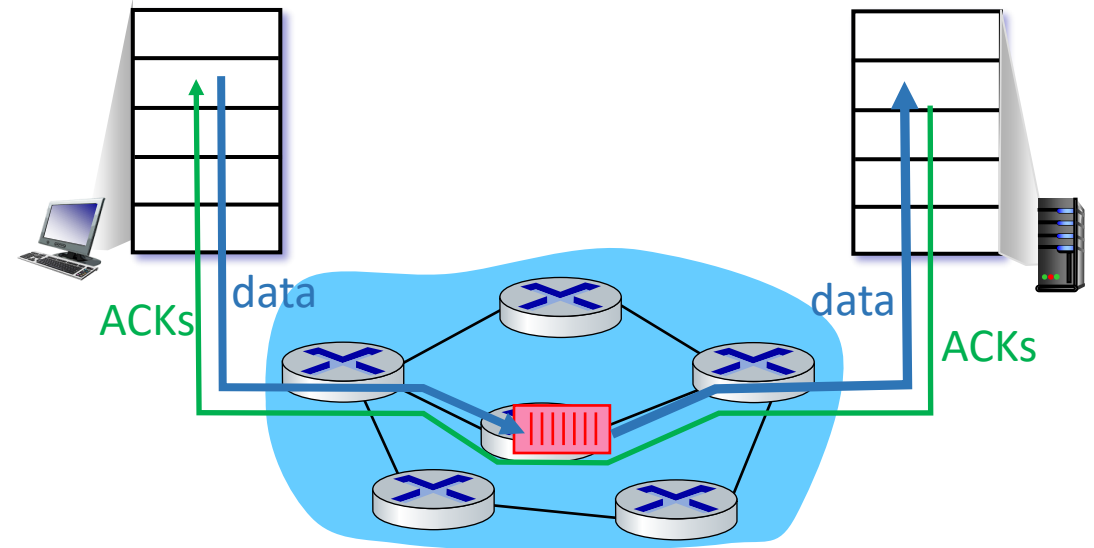
R: à medida que λ'_{in} cresce, todos pacotes azuis que chegam na fila superior são descartados, vazão azul $\rightarrow 0$



Abordagens para o controle de congestionamento

controle fim-a-fim:

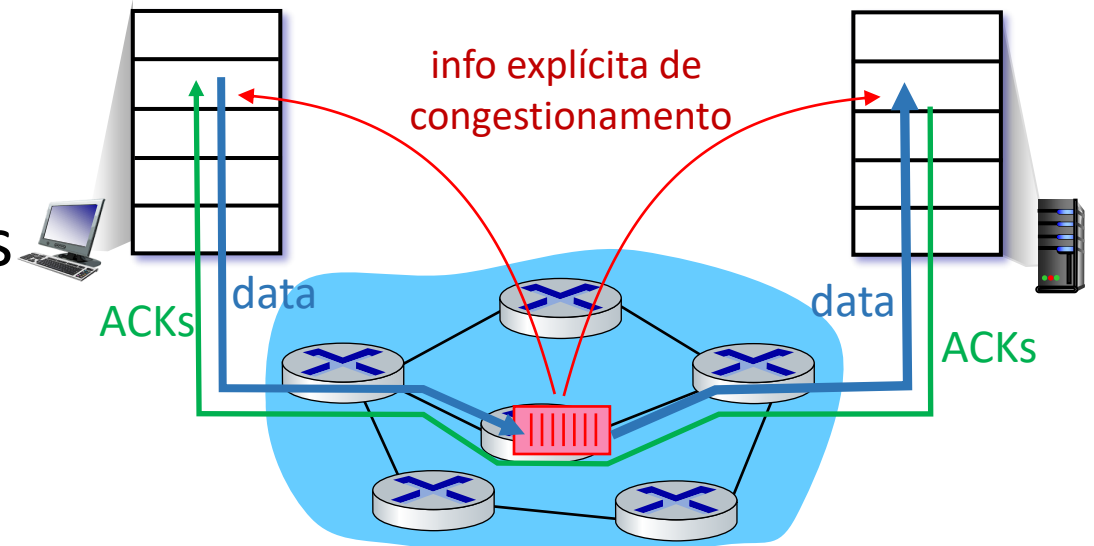
- sem realimentação explícita da rede
- congestionamento *inferido* a partir da perda, atraso observados
- abordagem usada pelo TCP



Abordagens para o controle de congestionamento

Controle de congestionamento assistido pela rede:

- roteadores proveem realimentação *direta* aos hospedeiros transmissores/receptores com fluxos passando pelo roteador congestionado
- pode indicar o nível do congestionamento ou informar explicitamente a taxa de transmissão
- protocolos TCP ECN, ATM, DECbit



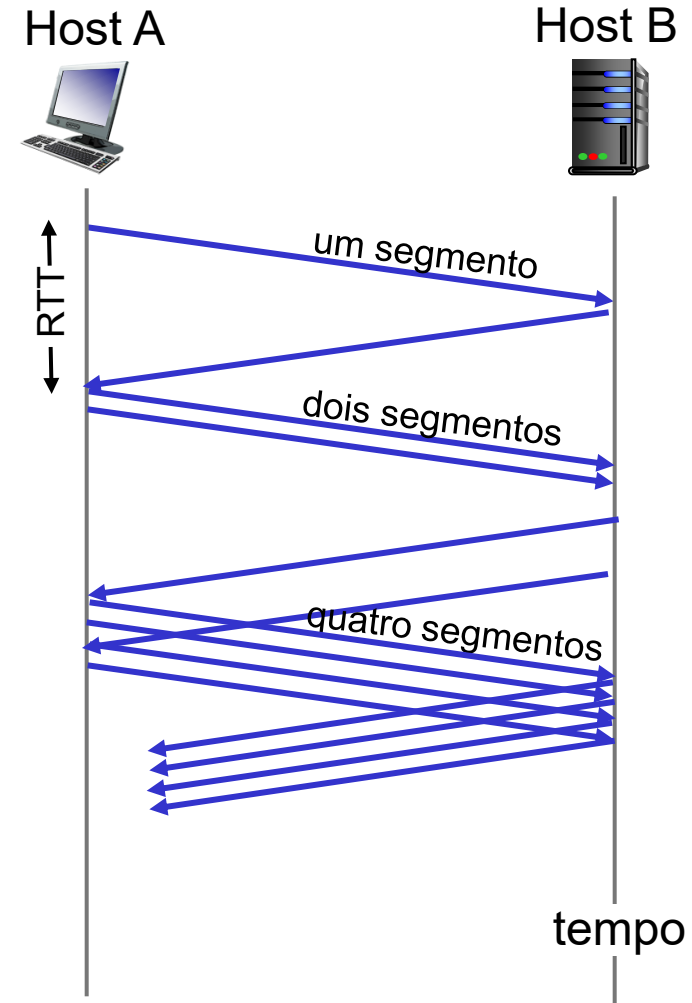
Capítulo 3: roteiro

- Serviços da camada de transporte
- Multiplexação e demultiplexação
- Transporte sem conexões: UDP
- Princípios da transferência confiável
- Transporte orientado a conexões: TCP
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte



TCP: partida lenta (*slow start*)

- no início da conexão, aumenta a taxa exponencialmente até o primeiro evento de perda:
 - inicialmente **cwnd** = 1 MSS
 - duplica **cwnd** a cada RTT
 - através do incremento de **cwnd** para cada ACK recebido
- *resumo*: taxa inicial é baixa mas cresce rapidamente de forma exponencial



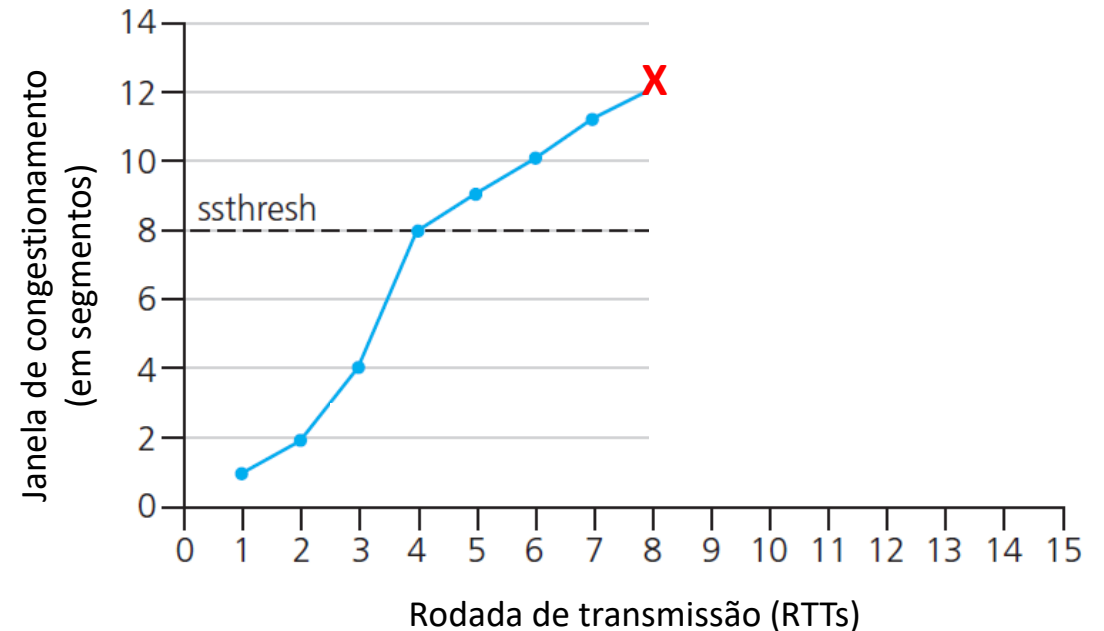
TCP: da partida lenta a evitar congestionamento

Q: quando o crescimento exponencial deve mudar para linear?

R: quando **cwnd** atingir 1/2 do seu valor antes do estouro do temporizador

Implementação:

- limiar variável **ssthresh**
- num evento de perda, **ssthresh** passa a ser 1/2 de **cwnd** imediatamente antes do evento de perda

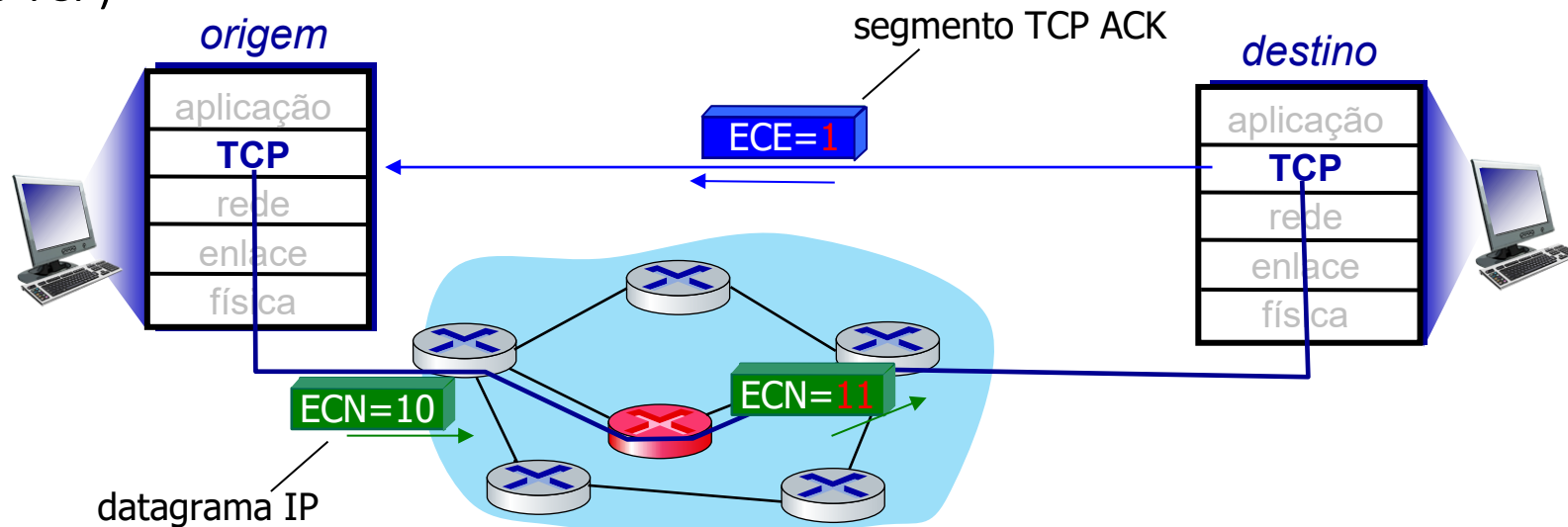


* Confira os exercício interativos online para mais exemplos: http://gaia.cs.umass.edu/kurose_ross/interactive/

Notificação explícita de congestionamento (ECN)

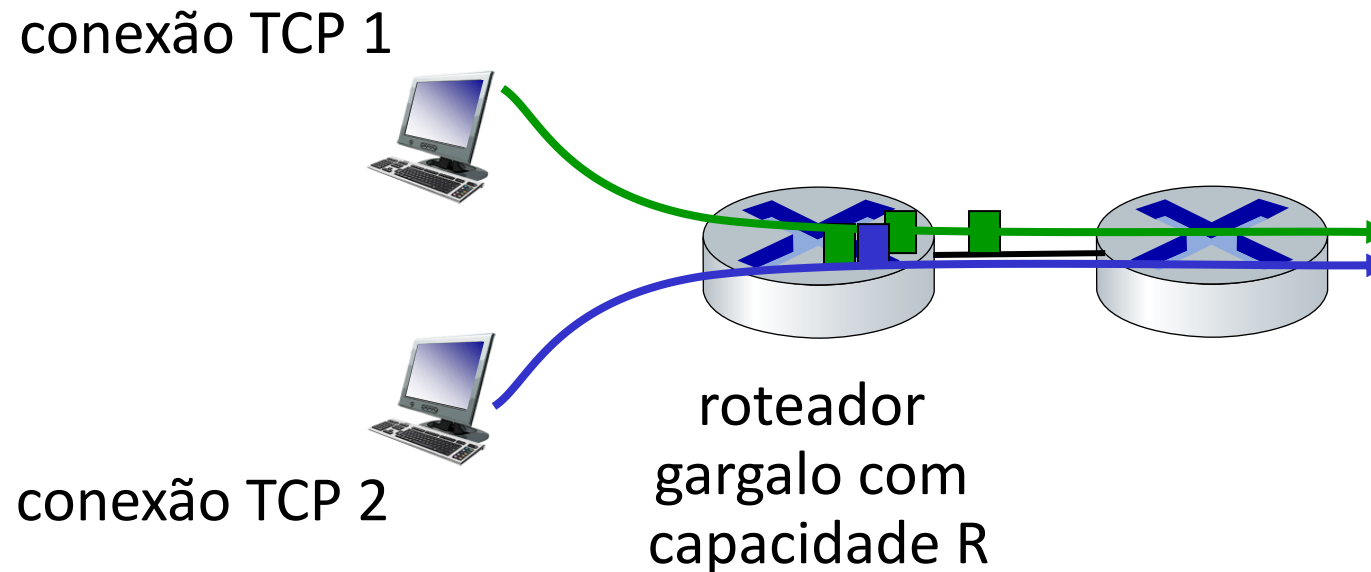
redes TCP frequentemente implementam controle de congestionamento *assistido pela rede*:

- dois bits no cabeçalho IP (campo ToS) marcado *pelo roteador* para indicar congestionamento
 - *política* para determinar a marcação escolhida pelo operador da rede
- indicação de congestionamento transportada até o destino
- destino seta o bit ECE no segmento de ACK para notificar o transmissor sobre o congestionamento
- envolve tanto o IP (marcação do bit ECN no cabeçalho IP) quanto o TCP (marcação dos bits C, E no cabeçalho TCP)



TCP: equidade (*fairness*)

Objetivo da equidade: se K sessões TCP compartilham o mesmo enlace de gargalo com largura de banda R , cada um deve obter uma taxa média de R/K



Todas as apps de rede devem ser “justas”?

Justiça e UDP

- apps multimídia em geral não usam TCP
 - não querem que a taxa seja limitada pelo controle de congestionamento
- em geral usam UDP:
 - enviam áudio/vídeo em taxas constantes, toleram perda de pacotes
- não há nenhuma “polícia Internet” policiando o uso do controle de congestionamento

Justiça, conexões TCP em paralelo

- aplicação pode abrir *múltiplas* conexões em paralelo entre dois hospedeiros
- navegadores web fazem isso, ex., enlace de taxa R com 9 conexões existentes:
 - nova app pede 1 TCP, obtém taxa $R/10$
 - nova app pede 11 TCPs, obtém $R/2$

Capítulo 3: roteiro

- Serviços da camada de transporte
- Multiplexação e demultiplexação
- Transporte sem conexões: UDP
- Princípios da transferência confiável
- Transporte orientado a conexões: TCP
- Princípios de controle de congestionamento
- Controle de congestionamento no TCP
- Evolução da funcionalidade da camada de transporte



Evoluindo a funcionalidade da camada de transporte

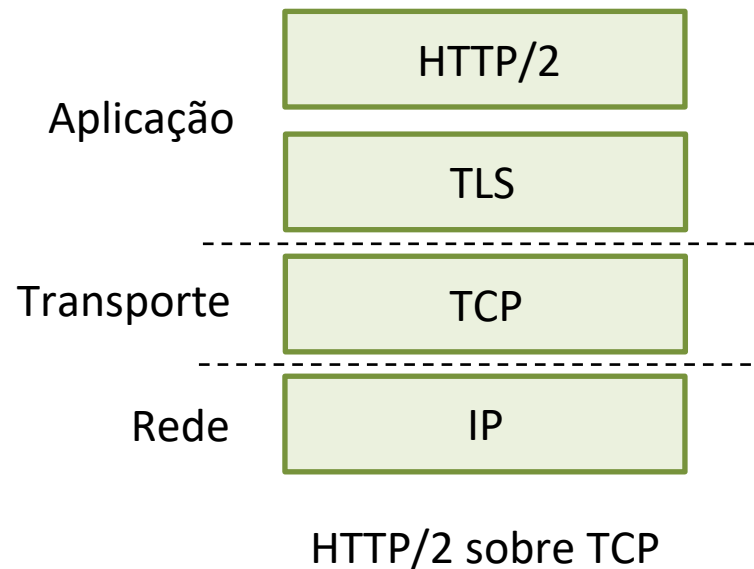
- TCP, UDP: principais protocolos de transporte por 40 anos
- diferentes “sabores” de TCP desenvolvidos, para cenários específicos:

Cenário	Desafios
Tubos longos e gordos (transferência de muitos dados)	Muitos pacotes “em trânsito”; perda “desliga” o tubo
Redes Wireless	Perda devido a enlaces wireless com ruído, mobilidade; TCP trata como perda por congestionamento
Enlaces com longos atrasos	RTTs extremamente longos
Redes de Data center	Sensível à latência
Fluxos de tráfego de fundo	Fluxos TCP de baixa prioridade, de fundo

- Mover funções da camada de transporte para a camada de aplicação, em cima do UDP
 - HTTP/3 (RFC 9114 – Junho 2022): QUIC (RFC 9000 – Maio 2021)

QUIC: *Quick UDP Internet Connections*

- protocolo de camada de aplicação, acima do UDP
 - aumenta o desempenho do HTTP
 - implantado em muitos servidores Google, apps (Chrome, app YouTube móvel)

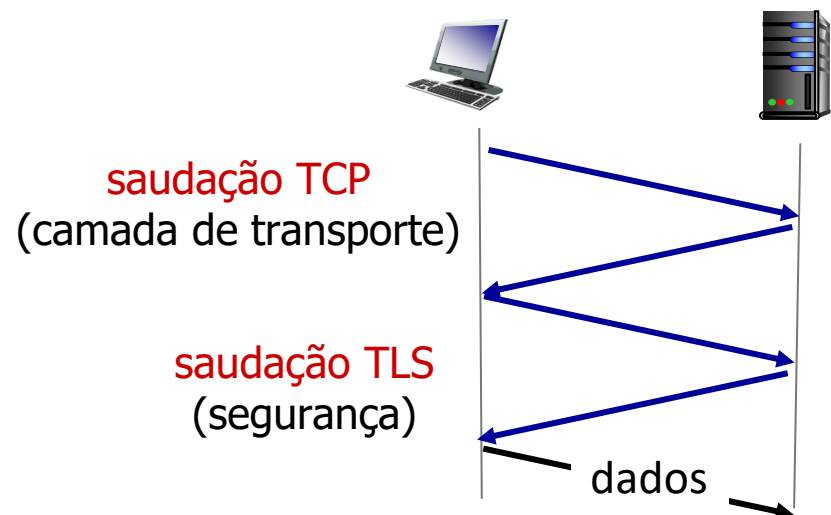


QUIC: *Quick UDP Internet Connections*

adota abordagens que estudamos neste capítulo para estabelecimento de conexão, controle de erro, controle de congestionamento

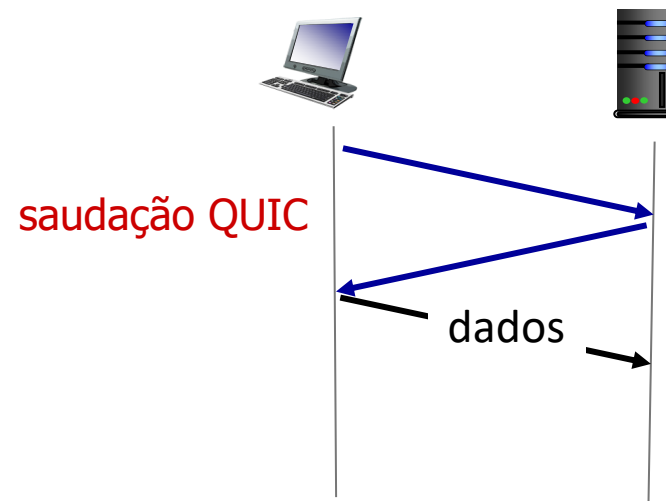
- **controle de erro e de congestionamento:** “Leitores familiarizados com a detecção de perda e controle de congestionamento do TCP encontrarão aqui algoritmos semelhantes aos do TCP.” [da especificação do QUIC]
- **estabelecimento de conexão:** confiabilidade, controle de congestionamento, autenticação, criptografia, estado estabelecido em um RTT
- múltiplos “fluxos” de aplicação multiplexados sobre uma única conexão QUIC
 - separa transferência confiável de dados, segurança
 - controle de congestionamento comum

QUIC: estabelecimento de conexão



TCP (estado da confiabilidade, controle de congestionamento) + TLS (estado da autenticação, criptografia)

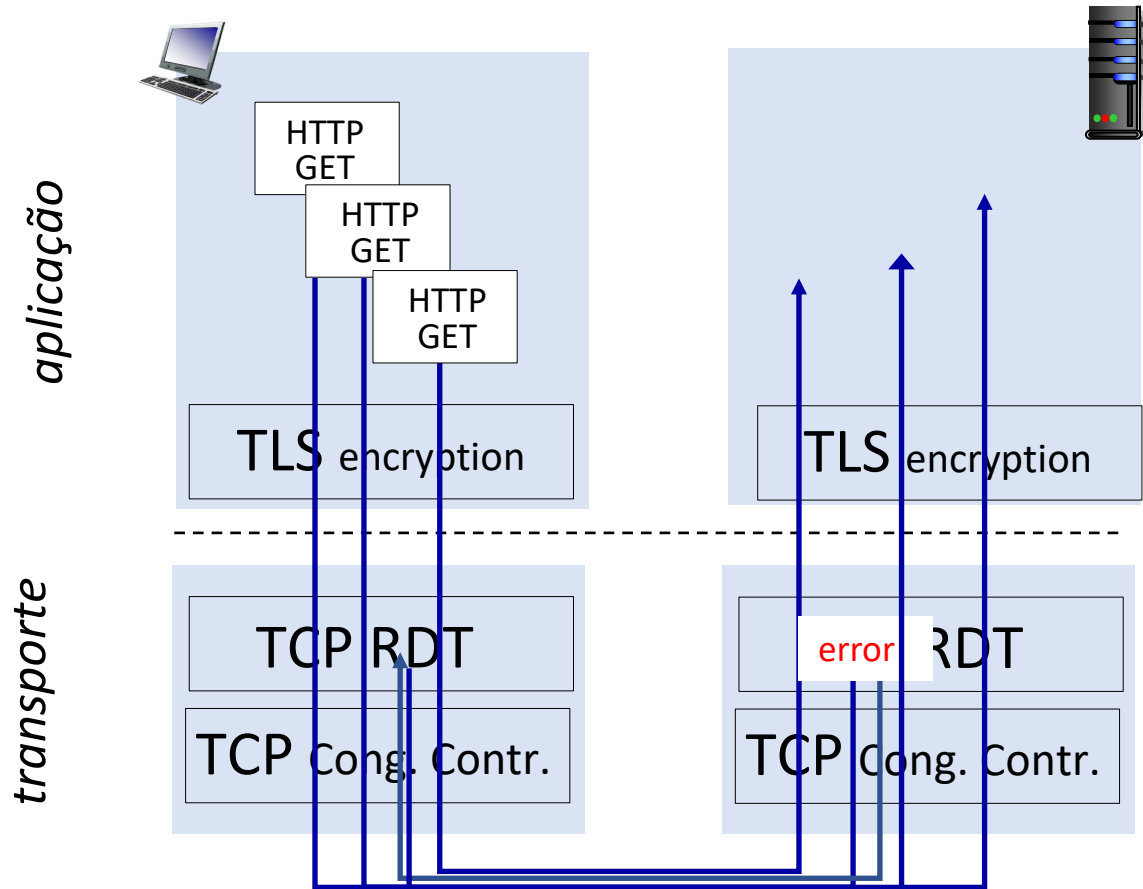
- 2 saudações em série



QUIC: estado da confiabilidade, controle de congestionamento, autenticação, criptografia

- 1 saudação

Fluxos QUIC: paralelismo, sem bloqueio HOL



(a) HTTP 1.1

Capítulo 3: resumo

- Princípios por trás dos serviços da camada de transporte:
 - multiplexação, demultiplexação
 - transferência (confiável) de dados
 - controle de fluxo
 - controle de congestionamento
- instanciação, implementação na Internet
 - UDP
 - TCP

A seguir:

- deixaremos a “borda” da rede (camadas aplicação, transporte)
- Entraremos no “núcleo” da rede
- dois capítulos sobre a camada de rede:
 - plano de dados
 - plano de controle